Machine Translated by Google

O'REILLY®

Guia Técnico

Entendimento ETL

Pipelines de dados para Arquiteturas de dados modernas

EDIÇÃO ATUALIZADA



Matt Palmer

Machine Translated by Google

databricks

Lakeflow

Built to simplify ETL

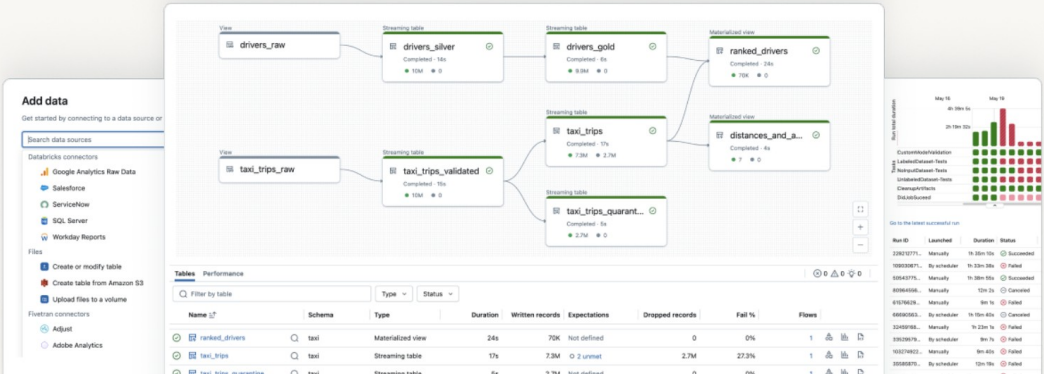
Databricks Lakeflow is built to address the key challenges data engineering teams face today. As an end-to-end data engineering solution with built-in data intelligence, Lakeflow unifies and simplifies data ingestion, transformation and orchestration by offering a singular approach to data pipeline creation, empowering data professionals at every stage of ETL.

Ingest
with Lakeflow Connect

Transform
with Lakeflow Declarative Pipelines

Orchestrate
with Lakeflow Jobs

Learn more



Machine Translated by Google

EDIÇÃO ATUALIZADA

Entendendo o ETL

Pipelines de dados para dados modernos Arquiteturas

Matt Palmer

O'REILLY®

Machine Translated by Google

Entendendo ETL (Edição Atualizada) por Matt Palmer

Copyright © 2025 O'Reilly Media, Inc. Todos os direitos reservados.

Publicado por O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

Os livros da O'Reilly podem ser adquiridos para fins educacionais, comerciais ou promocionais de vendas. Edições online também estão disponíveis para a maioria dos títulos (http://oreilly.com). Para mais informações, entre em contato com nosso departamento de vendas corporativas/institucionais: 800-998-9938 ou corporate@oreilly.com.

Editor de Aquisições: Aaron Black;
Editor de Desenvolvimento: Gary O'Brien; **Editor de Produção:** Kristen Brown; **Revisor de Texto:** nSight, Inc.
Revisor: M & R Consultants Corporação

Designer de Interiores: David Futato
Designer da capa: Ellie Volckhausen
Ilustradora: Kate Dullea

Março de 2024: Primeira Edição
Agosto de 2025: Edição atualizada

Histórico de revisões da edição atualizada
22/08/2025: Primeiro lançamento

Consulte http://oreilly.com/catalog/errata.csp?isbn=9798341665118 para obter detalhes sobre a versão.

O logotipo da O'Reilly é uma marca registrada da O'Reilly Media, Inc. Understanding ETL (Updated Edition), a imagem da capa e a identidade visual relacionada são marcas comerciais da O'Reilly Media, Inc.

As opiniões expressas nesta obra são de responsabilidade do autor e não representam as opiniões da editora. Embora a editora e o autor tenham se empenhado de boa-fé para garantir a precisão das informações e instruções contidas nesta obra, eles se eximem de toda responsabilidade por erros ou omissões, incluindo, sem limitação, a responsabilidade por danos resultantes do uso ou da confiança nesta obra. O uso das informações e instruções contidas nesta obra é de sua inteira responsabilidade. Caso algum exemplo de código ou outra tecnologia contida ou descrita nesta obra esteja sujeita a licenças de código aberto ou direitos de propriedade intelectual de terceiros, é sua responsabilidade garantir que seu uso esteja em conformidade com tais licenças e/ou direitos.

Este trabalho faz parte de uma colaboração entre a O'Reilly e a Databricks. Consulte nossa declaração de independência editorial.

979-8-341-66508-8

[LSI]

Machine Translated by Google

Índice

Introdução. vii

1. Ingestão de dados. 1

Ingestão de dados — agora versus antes 2
Fontes e Alvos 2
Considerações sobre a ingestão 10
Escolhendo uma solução 19

2. Transformação de Dados. 25

O que é transformação de dados? 25
Construindo uma solução de transformação: 33
O futuro da transformação de dados 40

3. Orquestração de Dados. 43

O que é orquestração de dados? 44
Ferramentas de orquestração 47
de dados: padrões de design e melhores 54
práticas; o futuro da orquestração de dados. 59

4. Problemas e resolução de problemas em dutos. 61

Manutenibilidade 62
Monitoramento e avaliação comparativa 63
de erros, 68
aprimorando fluxos de trabalho. 70

Machine Translated by Google

5. Eficiência e escalabilidade. 77
Definição de Eficiência e Escalabilidade 78
Entenda seu ambiente 80
Eficiência do processo 87
Conclusão 92

Conclusão. 93

Machine Translated by Google

Introdução

Seja qual for o seu cargo, engenheiro de dados ou outra profissão voltada para dados (estamos de olho em vocês, analistas e cientistas!), é provável que você já tenha ouvido falar em ETL. Há uma boa chance de que o ETL faça parte da sua vida, mesmo que você não saiba!

A sigla ETL significa extrair, transformar e carregar, e é usada para descrever o fluxo de trabalho fundamental que a maioria dos profissionais de dados executa: extrair dados de um sistema de origem, modificá-los para atender às suas necessidades e carregá-los em um destino.

Quer ajudar os líderes de produto a tomar decisões baseadas em dados? O ETL cria as tabelas essenciais para seus relatórios. Quer treinar a próxima versão do modelo de aprendizado de máquina da sua equipe? O ETL cria conjuntos de dados de alta qualidade. Está buscando mais estrutura e rigor nas políticas de armazenamento da sua empresa para atender aos requisitos de conformidade?
O ETL trará processos, linhagem e observabilidade aos seus fluxos de trabalho.

Se você deseja fazer qualquer coisa com dados, precisa de um processo ou fluxo de trabalho confiável. Essa verdade fundamental se aplica desde as cargas de trabalho clássicas de Business Intelligence (BI) até os avanços de ponta, como modelos de linguagem de grande escala (LLMs) e Inteligência Artificial (IA).

O Admirável Mundo Novo da IA

O mundo dos dados já viu muitas tendências surgirem e desaparecerem; algumas transformaram o setor, enquanto outras se revelaram modismos passageiros. A mais recente é, sem dúvida, a IA generativa (GenAI).

A cada esquina, surge a conversa sobre IA, LLMs e agentes. Esse fascínio recente pela IA, em grande parte impulsionado pelo lançamento do ChatGPT da OpenAI, vai além do interesse da mídia e se estende a diversos setores.

Machine Translated by Google

pesquisadores — agora é visto por muitos como um investimento estratégico essencial... e quem quer ficar para trás?

O verdadeiro valor dos Modelos de Aprendizagem Baseados em Lógica (LLMs) reside nos embeddings ou no ajuste fino dos modelos em conjuntos de dados limpos e selecionados. Essas técnicas permitem a criação de modelos com conhecimento específico do domínio, evitando erros comuns, como alucinações.

É claro que embeddings significativos são derivados de, você adivinhou, conjuntos de dados limpos. Nesse sentido, a IA é construída sobre a transformação de dados.
Seu sucesso depende fortemente da capacidade de criar conjuntos de dados consistentes e de alta qualidade em grande escala. Os dados precisam ser movidos, modificados e mesclados em um único local — pode-se dizer extraídos, transformados e carregados.

Isso mesmo — até mesmo a tecnologia mais inovadora tem suas raízes no ETL.

Um cenário de dados em transformação

Além do recente crescimento da Inteligência Artificial Geral (GenAI), outras tendências remodelaram o cenário de dados na última década. Uma dessas tendências é a crescente importância dos dados de fluxo contínuo (streaming). As empresas agora geram grandes quantidades de dados em tempo real por meio de sensores, sites, aplicativos móveis e muito mais. Essa mudança exige a ingestão e o processamento de dados em tempo real para a tomada de decisões imediatas. Os engenheiros de dados, portanto, são desafiados a ir além do processamento em lote tradicional para construir e gerenciar pipelines contínuos capazes de lidar com grandes volumes de dados de fluxo contínuo.

Outro desenvolvimento notável é o surgimento das arquiteturas de data lakehouse. O data lakehouse representa um conceito inovador, que busca combinar as capacidades de data warehouses e data lakes.
Aproveitando novas tecnologias de armazenamento como o Delta Lake, que aprimoram a confiabilidade e o desempenho dos data lakes, o modelo lakehouse combina o armazenamento escalável e econômico dos data lakes com o processamento eficiente de transações dos data warehouses. Essa fusão permite a execução tanto de cargas de trabalho de IA (normalmente tratadas em data lakes) quanto de cargas de trabalho analíticas (geralmente realizadas em data warehouses) em uma única estrutura. Essa integração reduz significativamente as complexidades associadas à manutenção de arquiteturas paralelas, à garantia de governança de dados consistente e ao gerenciamento da duplicação de dados.

Machine Translated by Google

Embora o ETL seja um conceito consolidado na gestão de dados, sua relevância permanece inabalável no cenário atual. Uma questão crucial agora é como os processos de ETL podem se adaptar para abranger tanto dados em lote quanto dados de fluxo contínuo, e como podem ser integrados de forma eficaz em uma arquitetura de data lake. Este guia visa esclarecer esses aspectos, ajudando você a compreender o ETL à luz dessas tendências em evolução.

E quanto ao ELT (e outros sabores)?

Ao se aprofundar em engenharia de dados, você pode se deparar com termos como ELT, além de ETL. Você pode estar pensando: "Nossa, esses caras deveriam contratar um revisor", mas fique tranquilo, eles são, na verdade, diferentes.
termos.

A principal diferença no ELT reside na sequência: no ELT, tudo é carregado em um recurso de preparação (staging) e, em seguida, transformado nas etapas subsequentes. O ELT tornou-se cada vez mais a norma, suplantando o ETL em muitos cenários — como muitos dizem, "armazenamento é barato". O termo "ETL" tem sido amplamente utilizado por tanto tempo (desde a criação dos próprios bancos de dados) que ainda é comumente usado, mesmo quando ELT é mais preciso.
Estamos agora na era do "armazenar primeiro, agir depois", facilitada pela redução dos custos de armazenamento em nuvem e pela facilidade de geração de dados.

Para análise, a retenção de todos os dados potencialmente úteis é fundamental. Avanços tecnológicos como a arquitetura Medallion e o Data Lake House dão suporte a essa abordagem com recursos como evolução simplificada de esquemas e viagens no tempo. Discutiremos esses e outros aspectos ao longo deste guia.

Embora usemos predominantemente o termo "ETL", é importante observar que os princípios e considerações discutidos são aplicáveis tanto ao ETL quanto ao ELT, bem como a outras variações, como o ETL reverso — a prática de ingerir dados limpos de volta em ferramentas de negócios a partir do data warehouse ou lakehouse. Não, ETL reverso ÿ LTE, e sim, isso é confuso, mas estamos nos desviando do assunto.

Independentemente de o termo "ETL" descrever precisamente o seu processo atual ou não, compreender os fundamentos da ingestão, transformação e orquestração de dados continua sendo crucial. Isso também se estende às melhores práticas em áreas como observabilidade, solução de problemas, escalabilidade e otimização. Esperamos que este guia seja um recurso valioso, independentemente da metodologia específica de processamento de dados que você utiliza.

Machine Translated by Google

Aprendizagem online da O'Reilly

> Há mais de 40 anos, a O'Reilly Media fornece treinamento, conhecimento e informações sobre tecnologia e negócios para ajudar as empresas a terem sucesso.

Nossa rede exclusiva de especialistas e inovadores compartilha seu conhecimento e experiência por meio de livros, artigos e nossa plataforma de aprendizado online. A plataforma de aprendizado online da O'Reilly oferece acesso sob demanda a cursos de treinamento ao vivo, trilhas de aprendizado aprofundadas, ambientes de programação interativos e uma vasta coleção de textos e vídeos da O'Reilly e de mais de 200 outras editoras. Para mais informações, acesse https://oreilly.com.

Como entrar em contato conosco

Por favor, envie comentários e perguntas sobre este livro à editora:

O'Reilly Media, Inc.
141 Stony Circle, Suite 195 Santa
Rosa, CA 95401
800-889-8969 (nos Estados Unidos ou Canadá) 707-827-7019
(internacional ou local) 707-829-0104 (fax)
support@oreilly.com
https://oreilly.com/about/
contact.html

Temos uma página web para este livro, onde listamos erratas, exemplos e quaisquer informações adicionais. Você pode acessar esta página em https://oreil.ly/understandingETL.

Para notícias e informações sobre nossos livros e cursos, visite https://oreilly.com.

Encontre-nos no LinkedIn: https://linkedin.com/company/oreilly-media.

Assista aos nossos vídeos no YouTube: https://youtube.com/oreillymedia.

Machine Translated by Google

Agradecimentos

Embora todos nós estejamos sobre os ombros de gigantes, este guia em particular não teria sido possível sem a mentoria, a ajuda e o apoio de algumas pessoas muito dedicadas e atenciosas.

Primeiramente, agradeço aos meus parceiros da O'Reilly e da Databricks: Aaron Black, que me deu a oportunidade de escrever; Gary O'Brien, que foi um editor de desenvolvimento excepcional (e confidente!); Ori Zohar, que ajudou a moldar o guia como um todo; e Sumit Makashir e Pier Paolo Ippolito por suas excelentes e atentas revisões técnicas.

Agradeço a Zander Matheson pela ajuda na compreensão de streaming e processamento de fluxos. Além de desenvolver uma ferramenta incrível (Bytewax), Zander tem sido um grande amigo e um guru de dados em geral.

Agradeço a Aleks Tordova e à equipe da Coalesce, que se uniram para escrever meu primeiro guia e me proporcionaram inúmeras oportunidades de aprendizado e crescimento.

Agradeço à minha família, que me deu apoio incondicional em minha jornada — tanto na área de dados quanto na vida — apesar das minhas falhas, peculiaridades e palhaçadas em geral. Obrigado, Jasmine, Violet e Paul (e aos meus cachorros, Enzo e Rocky!).

Em seguida, agradeço a amigos incríveis que me apoiaram durante minha mudança de cidade, meu novo emprego, a escrita deste guia e a minha jornada de autodescoberta. Muitas mensagens de texto, conversas no Slack, telefonemas e memes me ajudaram a superar os momentos difíceis.
Em ordem alfabética, agradeço a JulieAnn, Kandace, Rob, Srini e Tyson.

Por fim, agradeço à comunidade de dados. Aos indivíduos que contribuem para o código aberto e apresentam em conferências, aos profissionais que se dedicam diariamente a aprimorar seus conhecimentos, aos educadores/mentores que nos impulsionam como área de pesquisa e a todos os autores cujos textos, ideias e conteúdo nos ajudaram a chegar onde estamos hoje: mal posso esperar para ver o que conquistaremos a seguir!

Machine Translated by Google

Machine Translated by Google

CAPÍTULO 1

Ingestão de dados

A ingestão de dados, em essência, envolve a transferência de dados de uma fonte para um destino designado. Seu principal objetivo é inserir os dados em um ambiente preparado para armazenamento, processamento, análise e inteligência artificial/aprendizado de máquina (IA/ML). Embora grandes organizações possam se concentrar na movimentação interna de dados (entre equipes), para a maioria de nós, a ingestão de dados enfatiza a extração de dados de fontes externas e seu direcionamento para destinos internos.

Numa era em que os dados assumem importância central tanto nos negócios quanto no desenvolvimento de produtos, a relevância de dados precisos e oportunos não pode ser subestimada. Essa crescente dependência de dados deu origem a uma infinidade de "fontes" das quais as equipes extraem informações para aprimorar os processos de tomada de decisão, criar produtos excepcionais e realizar uma série de outras ações. Por exemplo, uma equipe de marketing precisaria coletar dados de diversas plataformas de publicidade e análise, como Meta, Google (incluindo Google Ads e Analytics), Snapchat, LinkedIn e Mailchimp.

No entanto, com o passar do tempo, as APIs e as fontes de dados sofrem modificações. Colunas podem ser adicionadas ou removidas, campos podem ser renomeados e novas versões podem substituir as desatualizadas. Lidar com alterações de uma única fonte pode ser viável, mas e quanto a gerenciar alterações de múltiplas fontes — cinco, dez ou até mesmo cem? O desafio urgente é: "Como uma equipe de dados em formação pode lidar de forma eficiente com essas diversas fontes de maneira consistente e expansível?" Como engenheiros de dados, como garantimos nossa reputação?

Machine Translated by Google

Garantir acesso a dados confiável e descomplicado, especialmente quando as demandas de cada departamento estão em constante crescimento?

Ingestão de dados — agora versus antes

Embora os princípios de ingestão permaneçam em grande parte os mesmos, muita coisa mudou. À medida que o volume, a velocidade e a variedade dos dados evoluem, nossos métodos também devem evoluir.

O setor passou por diversas mudanças para se adaptar a isso — a migração para a nuvem, do data warehouse para o data lake e para o lakehouse, além da simplificação das tecnologias de streaming, para citar algumas.
Isso se manifestou como uma mudança do processo de extração-transformação-carregamento (ETL).

Os fluxos de trabalho ETL (Extração, Carga e Transformação) para ELT (Extração, Carga e Transformação) têm como principal diferença o fato de que todos os dados agora são carregados em um sistema de destino. Discutiremos esses ambientes no contexto da transformação no Capítulo 2.

Evitamos ser muito rigorosos com os termos ETL e ELT; no entanto, gostaríamos de enfatizar que quase todos os fluxos de trabalho modernos de engenharia de dados envolvem o armazenamento temporário de quase todos os dados na nuvem.
A exceção notável são os casos em que centenas de trilhões de linhas de dados altamente granulares são processadas diariamente (por exemplo, Internet das Coisas [IoT] ou dados de sensores), onde faz sentido agregar ou descartar dados antes do armazenamento temporário.

Apesar das constantes mudanças, a verdade fundamental da extração é que os dados são retirados de uma fonte e gravados em um destino. Portanto, a discussão sobre extração deve se concentrar precisamente nisso.

Fontes e Alvos

Embora a maioria das pessoas associe a ingestão à extração, ela também está intimamente ligada ao carregamento; afinal, toda origem requer um destino. Neste guia, partimos do pressuposto de que você já possui um data warehouse ou data lake estabelecido; portanto, o armazenamento não será o foco principal deste capítulo. Em vez disso, destacaremos as melhores práticas para o preparo de dados e as características de implementações de armazenamento ideais.

Machine Translated by Google

Nossa missão é fornecer a você um conjunto de ferramentas para o projeto arquitetônico, tendo em mente que uma solução "perfeita" pode não existir. Vamos orientá-lo(a) em uma estrutura para avaliar fontes e desvendar as complexidades da ingestão de dados. Nossa abordagem de alto nível foi concebida para lhe dar uma visão panorâmica do cenário, permitindo que você tome decisões informadas e adequadas.

A Fonte

Nossa principal preocupação ao ingerir dados é a fonte e suas características. A menos que você tenha muita sorte, haverá muitas fontes. Cada uma deve ser avaliada separadamente para garantir recursos adequados e definir os critérios para sua(s) solução(ões) de ingestão.

Com o enorme volume de fontes de dados e a natureza das necessidades de negócios (raramente me pediram para remover fontes, mas adicionar uma é rotina), é muito provável que você encontre uma ou mais fontes que não se encaixam em uma única solução. Embora construir confiança leve semanas, meses e anos, ela pode ser perdida em um dia. A ingestão confiável e oportuna é fundamental. Então, o que é importante na escolha de uma fonte?

Analisando as

fontes: Como guia prático, adotamos a abordagem de apresentar perguntas comprovadas que o orientarão na compreensão dos dados de origem, tanto suas características quanto a forma como a empresa obterá valor com eles.

Recomendamos adotar uma postura bastante crítica: é sempre possível que os dados de origem não sejam necessários ou que uma fonte diferente se adeque melhor ao negócio. Você é o especialista em dados da sua organização e é sua responsabilidade verificar e confirmar as premissas. É normal tender à ação e à complexidade, mas é imprescindível considerarmos o essencialismo e a simplicidade.

Ao analisar as fontes, lembre-se de que provavelmente você estará trabalhando com engenheiros de software nos dados iniciais, mas as considerações posteriores são igualmente importantes. Negligenciá-las pode ser extremamente custoso, já que o erro pode não se manifestar até semanas após o início do trabalho.

Machine Translated by Google

Perguntas a fazer

Com quem vamos trabalhar?

Na era da inteligência artificial, priorizamos a inteligência real. A parte mais importante de qualquer fluxo de dados são as pessoas que ele atenderá. Quem são as partes interessadas envolvidas? Quais são suas principais motivações? OKRs (objetivos e resultados-chave) ou diretrizes organizacionais podem ser úteis para alinhar incentivos e agilizar o andamento dos projetos.

Como os dados serão utilizados?

Intimamente ligado ao "quem", o modo como os dados serão usados deve orientar amplamente as decisões subsequentes. Esta é uma forma de verificarmos os requisitos das partes interessadas e compreendermos o "problema por trás do problema" que elas estão tentando resolver. Recomendamos fortemente uma lista de requisitos técnicos de sim/não para evitar ambiguidades.

Qual é a frequência?

Como discutiremos em detalhes mais adiante, a maioria dos profissionais imediatamente pensa em processamento em lote versus processamento em fluxo contínuo. Qualquer dado pode ser processado em lote ou em fluxo contínuo, mas geralmente nos referimos às características dos dados que gostaríamos de processar em fluxo contínuo. Recomendamos que primeiro consideremos se os dados são limitados ou ilimitados — ou seja, se têm um fim (por exemplo, o conjunto de dados do Censo Americano de 2020) ou se são contínuos (por exemplo, dados de registro de um gabinete de fibra óptica).

Após considerarmos os limites, a frequência mínima disponível define um limite rígido para a frequência com que podemos extrair dados da fonte. Se uma API for atualizada apenas diariamente, haverá um limite rígido para a frequência dos seus relatórios. Os limites, a velocidade e os requisitos de negócio determinarão a frequência com que escolheremos extrair os dados.

Qual é o volume de dados esperado?

O volume de dados deixou de ser um fator limitante para a capacidade de armazenamento — afinal, "armazenamento é barato" e, embora o poder computacional possa ser elevado, está mais acessível do que nunca (por um fator de milhões; veja as Figuras 1-1 e 1-2). No entanto, o volume influencia diretamente a forma como escolhemos gravar e processar nossos dados, bem como a escalabilidade da solução desejada.

Machine Translated by Google

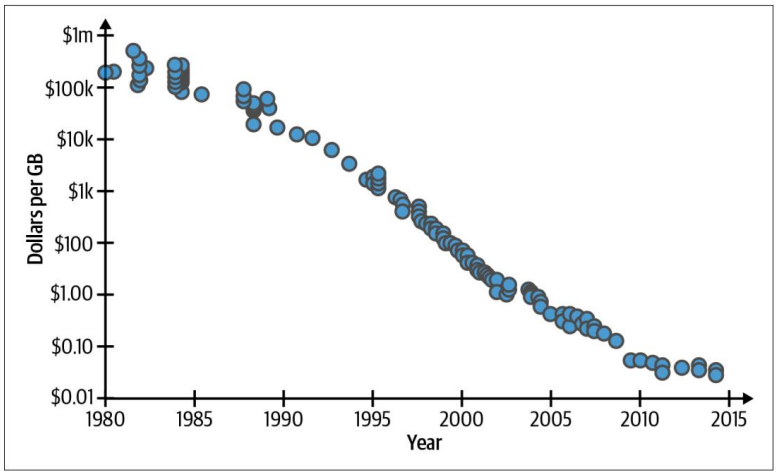


Figura 1-1. Custos de discos rígidos por GB, de 1980 a 2015 (Fonte: Matt Komorowski); os valores do eixo y estão em escala logarítmica.

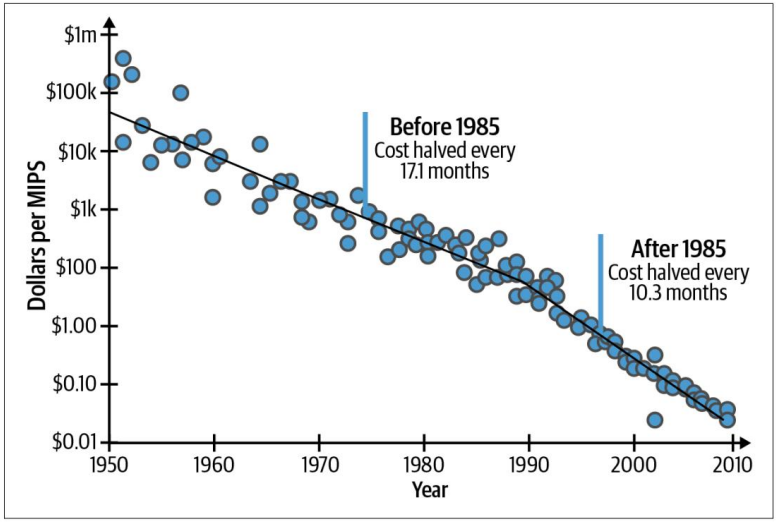


Figura 1-2. Custo computacional, milhões de instruções por segundo (MIPS) (Fonte: Field Robotics Center); os valores do eixo y estão em escala logarítmica.

Qual é o formato?

Embora eventualmente escolhamos um formato de armazenamento, o formato de entrada é uma consideração importante. Como os dados serão entregues? Será por meio de uma carga útil JSON (JavaScript Object Notation) através de uma API? Talvez um servidor FTP? Se você tiver sorte,

Machine Translated by Google

já reside em algum banco de dados relacional. Qual é o esquema? Existe um esquema? A infinidade de formatos de dados torna nosso trabalho interessante, mas também representa um desafio.

Qual é a qualidade?

A qualidade do conjunto de dados determinará em grande parte se alguma transformação será necessária. Como engenheiros de dados, é nossa responsabilidade garantir conjuntos de dados consistentes para nossos usuários. Os dados podem precisar ser bastante processados ou até mesmo enriquecidos com informações de fontes externas para complementar características ausentes.

Usaremos essas características para responder à nossa pergunta final:

Como os dados serão armazenados?

Como já mencionamos, este livro pressupõe um destino fixo para seus dados. Mesmo assim, há algumas considerações importantes no armazenamento de dados: armazenar ou não em etapas (será que isso realmente importa?), requisitos de negócios e adequação às partes interessadas são os pontos mais cruciais.

Lista de verificação de fontes

Para cada fonte que você encontrar, considere estas perguntas orientadoras.
Embora possa parecer assustador à medida que o número de fontes aumenta, lembre-se: esta não é uma tarefa de escrita. É uma estrutura para desvendar os desafios de cada fonte, ajudando você a esboçar soluções adequadas que atendam aos seus objetivos.

Embora possa parecer repetitivo, esse trabalho preparatório representa uma economia de tempo e recursos a longo prazo.

Pergunta	Exemplo:
Com quem vamos colaborar? Engenharia (Pagamentos)	
Como os dados serão utilizados?	Relatórios financeiros e planejamento estratégico trimestral
Existem múltiplas fontes?	Sim
Qual é o formato?	APIs semiestruturadas (Stripe e internas)
Qual é a frequência?	Por hora
Qual é o volume?	Aproximadamente 1.000 novas inscrições por dia, com um total existente de cerca de 100.000.
Que processamento é necessário?	Organização dos dados, como renomeação de colunas, e enriquecimento a partir de fontes suplementares.
Como os dados serão armazenados?	Armazenando dados em etapas em tabelas Delta via Databricks

Machine Translated by Google

O destino

Embora os sistemas de ponta a ponta exijam considerações hipotéticas que são apenas isso mesmo, hipotéticas, presumimos que a maioria dos leitores terá a tarefa de construir um pipeline em um sistema existente, onde a tecnologia de armazenamento já foi escolhida.

A escolha da tecnologia de armazenamento de dados está além do escopo deste guia, mas consideraremos brevemente os destinos, pois são de extrema importância para o valor total gerado por um sistema de dados. Portanto, ao analisar (ou considerar) um destino, recomendamos o uso de uma lista de verificação semelhante à utilizada para a origem. Normalmente, há muito menos destinos do que origens, então este exercício deve ser bem mais simples.

Analisando os destinos:

Um diferencial fundamental nos destinos é a priorização das partes interessadas. Os destinos possuem uma característica única: eles giram em torno das partes interessadas. Esses destinos impulsionam diretamente aplicações de BI, análise e IA/ML, ou as potencializam indiretamente ao lidar com dados pré-configurados, sem mencionar os aplicativos voltados para o usuário. Embora recomendemos a mesma lista de verificação, sugerimos adaptá-la um pouco mais para as partes interessadas, a fim de garantir que atenda às suas necessidades, enquanto se trabalha em prol dos objetivos de engenharia.

Reconhecemos plenamente que isso nem sempre é possível. Como engenheiro, seu papel é criar a solução mais adequada, mesmo que isso signifique optar por um meio-termo ou admitir que não há uma resposta definitiva. Apesar dos incríveis avanços tecnológicos, certos dilemas lógicos não têm soluções simples.

Preparação dos dados

ingeridos. Defendemos uma abordagem de data lake para a ingestão de dados. Isso implica ingerir a maior parte dos dados em sistemas de armazenamento em nuvem, como S3, Google Cloud Platform ou Azure, antes de carregá-los em um data warehouse para análise.

Um passo além é o lakehouse — que utiliza protocolos de armazenamento de dados, como o Delta Lake, que usa metadados para adicionar desempenho, confiabilidade e capacidade expandida. Os lakehouses podem até replicar algumas funcionalidades de um data warehouse; isso significa evitar a necessidade de carregar dados em um sistema de data warehouse separado. Adicionando uma camada de governança de dados,

Machine Translated by Google

Assim como o Unity Catalog, pode proporcionar melhor capacidade de descoberta, gerenciamento de acesso e colaboração para todos os ativos de dados em uma organização.

Uma prática comum e eficaz para o armazenamento temporário de dados é a utilização de formatos de arquivo baseados em Parquet e centrados em metadados, como Delta Lake, Apache Iceberg ou Apache Hudi. Fundamentados no Parquet — um formato colunar e compactado, projetado para grandes conjuntos de dados — esses formatos incorporam uma camada de metadados, oferecendo recursos como a possibilidade de viajar no tempo, conformidade com ACID (atomicidade, consistência, isolamento e durabilidade) e muito mais.

A integração desses formatos com a arquitetura Medallion, que processa dados em estágios em três camadas de qualidade distintas, garante a preservação de todo o histórico dos dados. Isso facilita a adição de novas colunas, a recuperação de dados perdidos e o preenchimento retroativo de dados históricos.

As nuances da arquitetura Medallion serão detalhadas em nosso capítulo sobre transformação de dados (Capítulo 2). Para a discussão atual, é pertinente considerar a viabilidade de direcionar todos os dados para uma “camada de preparação” dentro do provedor de armazenamento em nuvem escolhido.

Bancos de dados OLAP versus OLTP

A principal decisão em armazenamento de dados é escolher entre usar um banco de dados nativo da nuvem ou um banco de dados tradicional hospedado na nuvem — a principal diferença reside na natureza distribuída e na arquitetura orientada a colunas das soluções nativas da nuvem. Essas soluções são mais comumente conhecidas como OLAP (processamento analítico online) e OLTP (processamento de transações online).

- Os sistemas OLAP são projetados para processar grandes quantidades de dados rapidamente. Isso geralmente é feito por meio de processamento distribuído e uma arquitetura orientada a colunas. Os bancos de dados mais recentes, nativos da nuvem, são sistemas OLAP; as três principais soluções OLAP são Amazon Redshift, Google BigQuery e Snowflake. • Os sistemas OLTP são
- projetados para lidar com grandes quantidades de dados transacionais originados de múltiplos usuários. Isso geralmente assume a forma de um banco de dados orientado a linhas. Muitos sistemas de banco de dados tradicionais são OLTP: Postgres, MySQL, etc.

Os sistemas OLAP são mais comumente usados por equipes de análise e ciência de dados devido à sua velocidade, estabilidade e baixo custo de manutenção.
Aqui estão algumas considerações para a seleção de um data warehouse:

Machine Translated by Google

Plataforma de computação escolhida:

A integração dessas tecnologias depende em grande parte do restante da sua infraestrutura tecnológica. Por exemplo, se todas as ferramentas da sua organização estiverem no ecossistema da Amazon, o Redshift pode ser mais econômico e simples de implementar do que o Google BigQuery. Da mesma forma, o ecossistema Databricks oferece uma ampla gama de funcionalidades — desde armazenamento (Lakehouse, Databricks SQL) até governança (Unity Catalog) e computação (Apache Spark).

O BigQuery

oferece excelente suporte para dados semiestruturados e operações de aprendizado de máquina. O Redshift Spectrum provou ser uma ferramenta útil para criar tabelas externas a partir de dados no S3. O Databricks separa armazenamento e computação por meio do uso do Spark, Databricks SQL e Delta Lake. Cada data warehouse tem seus pontos fortes e fracos; estes precisam ser avaliados de acordo com o caso de uso da sua equipe.

Os

custos e suas estruturas de preços variam muito entre as plataformas. Infelizmente, a precificação também pode ser opaca — na maioria dos casos, você só saberá como um banco de dados será usado depois de já estar na plataforma.
O objetivo da precificação deve ser entender como usar um banco de dados para minimizar custos e qual seria o custo se essa estratégia fosse adotada. Por exemplo, em casos onde o custo está diretamente ligado à quantidade de dados analisados (BigQuery), o particionamento inteligente e a filtragem de consultas podem fazer uma grande diferença. Usar um lakehouse evita a redundância e a replicação de dados de um lake e de um data warehouse. Isso é especialmente importante, já que os data warehouses tendem a ter custos de armazenamento mais elevados e limitações.

Não haverá uma resposta direta sobre o custo, mas a pesquisa vale a pena, pois os armazéns podem ser caros, especialmente em grande escala.

Captura de dados

de alteração (CDC) é um padrão de projeto de engenharia de dados que captura e rastreia alterações em bancos de dados de origem para atualizar sistemas subsequentes. Em vez de carregar bancos de dados inteiros em lote, o CDC transfere apenas os dados alterados, otimizando a velocidade e a eficiência.
utilização de recursos.

Machine Translated by Google

Essa técnica é crucial para análises em tempo real e armazenamento de dados, pois garante que os dados nos sistemas de destino estejam atualizados e sincronizados com a fonte. Ao permitir atualizações incrementais, o CDC aprimora a disponibilidade e a consistência dos dados em todo o pipeline de dados. Simplificando, como Joe Reis e Matt Housley explicam em Fundamentos de Engenharia de Dados (O'Reilly, 2022), “o CDC… é o processo de ingestão de alterações de um sistema de banco de dados de origem”.

A CDC torna-se importante em análises e padrões de engenharia — como a criação de tabelas de Dimensão de Alteração Lenta (SCD) tipo 1 e 2, um processo que pode ser desnecessariamente complexo e demorado.
Escolher plataformas ou soluções que ofereçam suporte nativo ao CDC pode agilizar tarefas comuns, permitindo que você se concentre no que é mais importante. Um exemplo é o Lakeflow Declarative Pipelines (LDP) no Dataÿbricks, que oferece suporte nativo para SCD tipo 1 e 2 em pipelines de lote e de fluxo contínuo.

Lista de verificação do destino

Segue um exemplo de lista de perguntas a serem consideradas ao escolher um destino:

Pergunta	Exemplo
Com quem iremos colaborar?	Recursos Humanos.
Como os dados serão utilizados?	Entenda como a duração do contrato/permanência no cargo afeta os resultados financeiros.
Existem vários destinos?	Os dados são armazenados temporariamente no Delta Lake; as tabelas finais são criadas no Databricks.
Qual é o formato?	SQL.
Qual é a frequência?	Parquet no Delta Lake. SQL estruturado/semiestruturado no Databricks.
Qual é o volume?	Lote
Que processamento é necessário?	Aproximadamente 1.000 novas linhas por dia, com um conjunto existente de cerca de 100.000. Processamento downstream usando LDP. Modelos de ML e aplicações GenAI em Spark.
Como os dados serão armazenados?	Combinação de SQL do Databricks e tabelas externas no Delta Lake.

Considerações sobre a ingestão

Nesta seção, descrevemos as principais características dos dados. Embora esta lista não seja exaustiva e seja influenciada por contextos específicos, aspectos como frequência, volume, formato e processamento emergem como primordiais. preocupações.

Machine Translated by Google

Frequência. Já

mencionamos que a primeira consideração em relação à frequência deve ser os limites, ou seja, se o conjunto de dados é limitado ou ilimitado. Os limites e as necessidades de negócio determinarão a frequência de ingestão de dados — seja em formato de lote ou de fluxo contínuo. A Figura 1-3 mostra claramente a diferença entre os processos em lote e de fluxo contínuo; o fluxo contínuo captura os eventos à medida que ocorrem, enquanto o lote os agrupa, como o nome sugere.

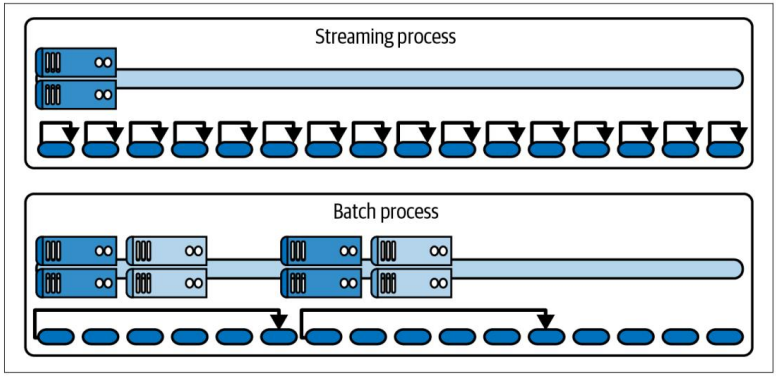


Figura 1-3. A latência é a propriedade que define os processos "em lote" ou "em fluxo contínuo"; além de um determinado limite de latência arbitrário, consideramos os dados como "em fluxo contínuo" (cortesia de Denny Lee).

Apresentaremos os métodos de ingestão em lote, micro-lote e streaming, juntamente com nossas considerações para ajudá-lo a selecionar a frequência mais adequada e uma solução de ingestão compatível.

Lote

O processamento em lote consiste em processar dados em partes, em vez de tudo de uma vez. Assim como um loop `for` que itera sobre uma fonte de dados, o processamento em lote envolve dividir um conjunto de dados limitado em partes menores e processar cada componente individualmente, ou processar conjuntos de dados ilimitados à medida que os dados chegam.

Micro-lotes

Um micro-lote é simplesmente uma forma de "reduzir a intensidade" do processamento em lote. Se uma ingestão em lote típica opera diariamente, um micro-lote pode funcionar a cada hora ou até mesmo a cada minuto. Claro, você pode dizer: "Em que ponto isso se torna apenas semântica? Um pipeline de micro-lote com latência de 100 ms me parece muito semelhante a um fluxo de dados em streaming."

Machine Translated by Google

Concordamos!

No restante deste capítulo (e do livro), nos referiremos a soluções de micro-lotes de baixa latência como soluções de streaming — sendo a mais óbvia o Spark Structured Streaming. Embora "tecnicamente" sejam de micro-lotes, a latência na ordem de centenas de milissegundos torna o Spark Structured Streaming efetivamente uma solução em tempo real.

Streaming

refere-se à leitura contínua de conjuntos de dados, sejam eles limitados ou ilimitados, à medida que são gerados. Embora não abordemos o streaming em detalhes, ele merece pesquisas adicionais. Para uma compreensão abrangente das fontes de dados de streaming, sugerimos explorar recursos como Streaming 101, Bancos de Dados de Streaming e, claro, Fundamentos de Engenharia de Dados.

Métodos. Os métodos comuns para transmissão de dados ilimitados incluem:

Segmentação
: por janelas: Segmentação de uma fonte de dados em blocos finitos com base em limites temporais.

Janelas fixas
: Os dados são essencialmente "micro-lotes" e lidos em pequenas janelas fixas para um destino.

Janelas de correr:
: Semelhantes às janelas fixas, mas com bordas sobrepostas.

Sessões:
: Janelas dinâmicas nas quais sequências de eventos são separadas por intervalos de inatividade — em sessões, a “janela” é definida pelos próprios dados.

Independente do
: tempo. Adequado para dados onde o tempo não é crucial, frequentemente utilizando cargas de trabalho em lote.

A Figura 1-4 demonstra a diferença entre janelas fixas, janelas deslizantes e sessões. É crucial diferenciar entre o tempo real do evento e o tempo de processamento, visto que podem surgir discrepâncias.

Machine Translated by Google

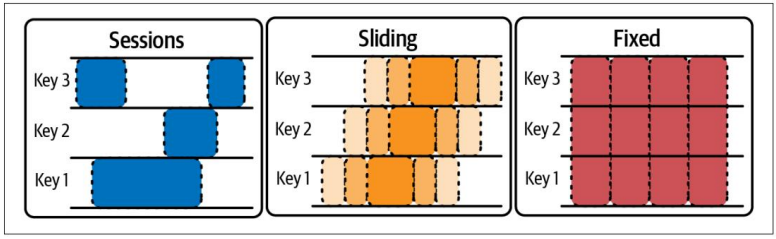


Figura 1-4. Janelas fixas, janelas deslizantes e sessões tratam a chegada de dados de streaming de maneiras bastante diferentes.

Serviços de mensagens. Quando falamos em “serviços de mensagens”, nos referimos a “camadas de transporte” ou sistemas para comunicação e transporte de dados em fluxo contínuo. É importante ressaltar que esta não é uma comparação direta; embora haja sobreposição entre esses serviços, muitos operam sob arquiteturas fundamentalmente diferentes, tornando discussões como “Kafka versus Pub/Sub” ou “Kinesis versus Redpanda” em grande parte irrelevantes.

Criado no
: LinkedIn em 2011, o Apache Kafka começou como um sistema de filas de mensagens, mas rapidamente evoluiu para uma plataforma de streaming distribuída. Embora o design do Kafka permita alta taxa de transferência e escalabilidade, sua complexidade inerente continua sendo um obstáculo para muitos.

Desenvolvido
: como uma alternativa ao Kafka, o Redpanda oferece desempenho semelhante com configuração e instalação simplificadas. O Redpanda é baseado em C++ em vez de Java e é compatível com as APIs do Kafka.

Pub/Sub
: é a oferta do Google Cloud para uma fila de mensagens dinâmica e durável. Ao contrário do Kafka, o Google Pub/Sub escala dinamicamente para lidar com cargas de trabalho variáveis. Em vez de trabalhar com "fluxos" e "fragmentos", o Pub/Sub opta por "tópicos" e "assinaturas". Um grande atrativo do Pub/Sub é a eliminação da maioria das tarefas de "manutenção" — ele é quase totalmente gerenciado.

Machine Translated by Google

Cinesis

O Kinesis é outro serviço robusto e totalmente gerenciado. Como um serviço da Amazon, o Kinesis oferece a óbvia facilidade de integração com outras ofertas da Amazon Web Services (AWS), além de proporcionar escalabilidade automática e processamento de dados em tempo real. Assim como o Pub/Sub, o Kinesis se destaca por sua natureza de serviço gerenciado, oferecendo uma carga operacional menor do que o Apache Kafka.

Mecanismos de processamento de fluxos. O processamento de fluxos consiste em analisar e agir sobre dados em tempo real (fluxos). Dada a longevidade do Kafka, as três ferramentas de processamento de fluxos mais populares e conhecidas são:

Apache Flink:

Um mecanismo de código aberto que processa continuamente conjuntos de dados ilimitados e limitados com tempo de inatividade mínimo.

O Apache Flink garante baixa latência por meio de computações em memória, oferece alta disponibilidade eliminando pontos únicos de falha e escala horizontalmente.

Apache Spark Structured Streaming é um

braço do ecossistema Apache Spark projetado para lidar com processamento de dados em tempo real. Ele traz a familiaridade e o poder das APIs DataFrame e Dataset do Spark para dados de streaming. O Structured Streaming pode ser uma opção atraente dada a popularidade do Apache Spark no processamento de dados e a onipresença do mecanismo em ferramentas como o Databricks.

Apache Kafka Streams: Uma

biblioteca construída sobre o Kafka que fornece recursos de processamento com estado, mas a dependência do Java pode ser limitante.

Simplificando o processamento de fluxos. Diversas soluções relativamente novas simplificam o processamento de fluxos, oferecendo clientes intuitivos, com foco em desempenho e ciclos de desenvolvimento simplificados.

Plataformas gerenciadas.

Tomando o Databricks como exemplo: aproveitar ferramentas como Lakeflow Declarative Pipelines ou simplesmente executar trabalhos do Spark Streaming no ambiente de execução do Databricks pode ser uma poderosa abstração da complexidade e simplificar drasticamente o processo de construção de sistemas de streaming.

Machine Translated by Google

Confluent Kafka é

uma tentativa de trazer as funcionalidades do Apache Kafka para o Python, embora ainda seja rudimentar em comparação com sua contraparte em Java. O Confluent Kafka é simplesmente uma biblioteca cliente, da mesma forma que o psycopg2 é uma biblioteca cliente para o Postgres.

Bytewax é

uma biblioteca que visa preencher essa lacuna, oferecendo uma maneira mais intuitiva e "pythonica" de lidar com processamento de fluxos, tornando-o mais acessível a uma gama maior de desenvolvedores. Construída em Rust, a Bytewax é altamente performática, mais simples que o Flink e possui ciclos de feedback mais curtos, além de fácil implantação e escalabilidade.

Ferramentas ainda mais recentes que buscam unificar o processamento de fluxos de dados — como o Apache Beam ou o Estuary Flow — ou combinar o processamento de fluxos diretamente com bancos de dados (bancos de dados de fluxo) estão ganhando popularidade. Recomendamos os livros "Streaming Systems" e "Streaming Databases" para uma análise mais aprofundada.

O cenário de streaming, embora complexo, tem apresentado avanços em termos de simplificação e facilidade de uso, especialmente quando se consideram plataformas gerenciadas, como o Databricks, e soluções de micro-lotes de baixa latência, como o Spark Structured Streaming.

Embora muitos considerem os dados em tempo real como o objetivo final, enfatizamos uma abordagem de "momento certo". Como em qualquer solução, a latência pode ser otimizada infinitamente, mas o custo da solução (e a complexidade) aumentará proporcionalmente. A maioria considerará a transição de dados diários ou semidiários para dados horários/subhorários uma solução perfeitamente aceitável.

Carga útil

O termo “carga útil” refere-se à mensagem propriamente dita que está sendo transmitida, juntamente com quaisquer metadados ou cabeçalhos usados para roteamento, processamento ou formatação dos dados. As cargas úteis de dados são inerentemente definidas de forma ampla, já que podem assumir praticamente qualquer formato imaginável. Nesta seção, discutiremos as características típicas das cargas úteis.

Volume

O volume é um fator crucial nas decisões de ingestão de dados, influenciando a escalabilidade das soluções de processamento e armazenamento. Ao avaliar o volume de dados, certifique-se de considerar fatores como:

Machine Translated by Google

Custos: Volumes maiores geralmente levam a custos mais altos, tanto em termos de armazenamento quanto de recursos computacionais. Certifique-se de alinhar o fator custo com seu orçamento e necessidades do projeto, incluindo custos de armazenamento/preparação associados a armazéns, lagos ou casas à beira de lagos, dependendo da sua solução.

Latência:
Dependendo se você precisa de ingestão de dados em tempo real, quase em tempo real ou em lote, a latência pode ser um fator crítico. O processamento em tempo real não só exige mais recursos, como também requer maior eficiência quando o volume aumenta repentinamente. Para qualquer volume de dados, certifique-se de que seus sistemas consigam lidar com os requisitos de latência.

Capacidade de processamento/
escalabilidade: É essencial conhecer a capacidade da ferramenta de ingestão de lidar com o grande volume de dados recebidos. Se a fonte de dados gera grandes quantidades de dados, a ferramenta de ingestão deve ser capaz de ingerir esses dados sem causar gargalos.

Retenção
Com grandes volumes de dados, as políticas de retenção tornam-se ainda mais importantes. Você precisará de uma estratégia para descartar dados antigos ou migrá-los para soluções de armazenamento de longo prazo mais econômicas. Além do armazenamento de dados antigos, a segurança e a recuperação (restauração) de dados perdidos também devem ser consideradas.

Para lidar com conjuntos de dados volumosos, o uso de formatos comprimidos como Apache Avro ou Parquet é crucial. Cada um oferece vantagens e limitações distintas, especialmente em relação à evolução do esquema. Para cada uma dessas considerações, certifique-se de olhar para o futuro — soluções viáveis podem se desintegrar rapidamente com aumentos exponenciais no volume de dados.

Estrutura e forma Os

dados variam amplamente em forma e estrutura, desde dados "estruturados" e relacionais bem definidos até dados "não estruturados" de formato mais livre.

É importante ressaltar que estrutura não é sinônimo de qualidade; significa apenas a presença de um esquema.

No cenário atual, impulsionado pela IA, o valor dos dados não estruturados está em franca expansão, à medida que os avanços em grandes modelos de linguagem e aprendizado de máquina nos permitem extrair insights valiosos desses dados. Apesar disso, os humanos

Machine Translated by Google

Possuem uma inclinação por dados estruturados quando se trata de análises aprofundadas, um fato sublinhado pela popularidade duradoura do SQL — Structured Query Language (Linguagem de Consulta Estruturada) — um elemento básico na análise de dados há quase meio século.

Dados **não estruturados.** Como já mencionamos, dados não estruturados são dados sem nenhum esquema ou estrutura predefinida. Na maioria das vezes, são representados como texto, mas outras formas de mídia também representam dados não estruturados. Vídeos, áudios e imagens possuem elementos que podem ser analisados numericamente. Dados não estruturados podem ser, por exemplo, o texto de um romance de Pierce Brown:

> “Um homem pensa que pode voar, mas tem medo de pular. Um pobre amigo o empurra por trás.” Ele olha para mim. “Um bom amigo pula junto.”

Esses dados frequentemente alimentam aplicações de aprendizado de máquina ou IA, o que reforça a necessidade de compreender os requisitos das partes interessadas de forma abrangente. Dada a complexidade do aprendizado de máquina, é fundamental entender como esses dados não estruturados serão utilizados antes de serem incorporados. Métricas como comprimento do texto ou tamanho não comprimido podem servir como medidas de forma.

Dados **semiestruturados.** Os dados semiestruturados situam-se algures entre os dados estruturados e os não estruturados — XML e JSON são dois formatos populares. Os dados semiestruturados podem assumir a forma de uma carga útil JSON:

```
'{"amigos": ["steph", "julie", "thomas", "tommy", "michelle", "tori",
"larry"]}'
```

À medida que as plataformas de dados continuam a amadurecer, também aumentará a capacidade de processar e analisar dados semiestruturados diretamente. O trecho de código a seguir mostra como analisar JSON semiestruturado no Google BigQuery para transformar uma lista em linhas de dados:

```
COM j_data COMO (
    SELECIONAR
        (
        JSON '{"amigos": ["steph", "julie", "thomas",
"tommy", "michelle", "tori", "larry"]}'
        ) COMO my_friends_json
), l_data AS ( SELECT

    JSON_EXTRACT_ARRAY( JSON_QUERY(j_data.my_friends_json, "$.friends"),
'$'
```

Machine Translated by Google

```
            ) como minha_lista_de_amigos
        A partir de j_data
)
    SELECIONAR
        meus_amigos
FROM l_data, UNNEST(l_data.my_friends_list) as meus_amigos ORDER BY RAND()
```

Em algumas situações, transferir o processamento de dados para a camada de análise é vantajoso, pois permite aos analistas e engenheiros de análise maior flexibilidade na forma como consultam e armazenam os dados. Dados semiestruturados em um data warehouse (ou acessados por meio de tabelas externas) oferecem flexibilidade em caso de mudanças de esquema ou dados faltantes, mantendo todos os benefícios da manipulação de dados tabulares e do SQL.

Ainda assim, devemos ter cuidado para validar adequadamente esses dados e garantir que a falta de dados não esteja causando erros. Muitas consultas inválidas resultam da consideração inadequada de dados NULL .

Descrever a estrutura de um JSON frequentemente envolve discutir chaves, valores e o número de elementos aninhados. Recomendamos fortemente o uso de ferramentas como JSONLint e JSON Crack para essa finalidade. Também existem extensões para o VS Code que permitem validar e formatar dados JSON/XML.

Dados **estruturados.** O ideal de dados estruturados são fontes organizadas de forma clara, com esquemas fixos e chaves imutáveis. Há mais de 50 anos, o SQL é a linguagem preferida para consultar dados estruturados. Ao armazenar dados estruturados, frequentemente nos preocupamos com o número de colunas e o comprimento da tabela (em linhas).
Essas características orientam nossa utilização de materialização, construções incrementais e, em conjunto, um banco de dados OLAP versus OLTP (orientado a colunas/linhas).

Embora muitos dados atuais careçam de estrutura, o SQL ainda é a ferramenta dominante para análise. Isso mudará? Possivelmente, mas, como mostramos, é mais provável que o SQL simplesmente se adapte para acomodar formatos semiestruturados. Embora ferramentas de consulta baseadas em linguagem tenham começado a surgir com os avanços da IA, o SQL geralmente serve como intermediário. Se o SQL desaparecer da análise de dados, provavelmente continuará existindo como uma API.

Machine Translated by Google

Formatar

Qual o melhor formato para os dados de origem? Embora JSON e CSV (valores separados por vírgula) sejam escolhas comuns, uma infinidade de considerações de formato podem surgir. Por exemplo, algumas transferências SFTP/FTP mais antigas podem chegar compactadas, exigindo uma etapa extra de extração.

O formato dos dados geralmente determina os requisitos de processamento e as soluções disponíveis. Embora uma ferramenta como o Airbyte possa se integrar perfeitamente a uma fonte CSV, ela pode apresentar problemas com um método de compressão personalizado ou uma codificação peculiar do Windows (acredite, isso acontece).

Se possível, recomendamos optar por formatos de dados conhecidos e populares. Assim como no conserto de um veículo, quanto mais popular o formato, mais fácil será encontrar recursos, bibliotecas e instruções. Ainda assim, em nossa experiência, lidar com um formato complexo é quase um rito de passagem, mas isso faz parte do que torna nosso trabalho divertido!

Variedade: É muito provável que você lide com múltiplas fontes e, portanto, com cargas úteis variadas. A variedade de dados desempenha um papel fundamental na escolha da sua solução de ingestão — ela não só deve ser capaz de lidar com tipos de dados distintos, como também deve ser flexível o suficiente para se adaptar a mudanças de esquema e formatos variados. A variedade torna a governança e a observabilidade particularmente desafiadoras, algo que abordaremos no Capítulo 5.

Não levar em consideração a variedade de dados pode resultar em gargalos, aumento da latência e um fluxo de trabalho desorganizado, comprometendo a integridade e a utilidade dos dados recebidos.

Escolhendo uma solução

As melhores ferramentas para sua equipe serão aquelas que dão suporte às suas fontes e alvos. Considerando as necessidades específicas de dados de cada organização, suas escolhas dependerão do contexto. Ainda assim, não podemos enfatizar o suficiente a importância de aproveitar o conhecimento de mentores, colegas e especialistas do setor. Embora as conferências possam ser caras, o conhecimento que elas proporcionam pode ser inestimável. Para quem tem um orçamento limitado, as comunidades de dados podem ser muito valiosas. Procure por elas em plataformas como Slack e LinkedIn e por meio de newsletters do setor.

Machine Translated by Google

Ao considerarmos uma solução de ingestão, pensamos em termos de considerações gerais e específicas da solução — as primeiras se aplicam a todas as ferramentas que analisaremos, e as últimas são específicas para a classe de ferramentas.

As considerações gerais incluem extensibilidade, custo de construção, custo de manutenção, custo de troca e outros aspectos do sistema.
preocupações.

As considerações específicas da solução dependem da classe de ferramentas, que geralmente assume uma das duas formas:

- Soluções declarativas ditam os resultados. Por exemplo: “Eu uso a interface do usuário do AWS Glue para criar um rastreador que processa dados sistematicamente” ou “Eu crio uma nova conexão Airbyte por meio de uma interface de usuário”.
- Soluções imperativas ditam ações. Por exemplo: "Eu crio uma função Lambda que chama a API do Stripe, codifica/decodifica dados e os carrega incrementalmente no Snowflake."

Cada uma dessas soluções tem seus prós e contras. Vamos discutir brevemente cada uma delas e apresentar o método que recomendamos para abordar a integração de dados.

Soluções Declarativas

Classificamos as soluções declarativas como legadas, modernas ou nativas, dependendo principalmente de sua adesão aos princípios da pilha de dados moderna (MDS), como infraestrutura como código, DRY (não se repita) e outras boas práticas de engenharia de software. As plataformas nativas diferem das duas primeiras — elas são integradas diretamente a um provedor de nuvem:

Ferramentas legadas como Talend, WhereScape e Pentaho podem ser consideradas. Elas possuem conectores robustos e contam com uma comunidade ativa e amplo suporte. No entanto, à medida que o cenário de dados evolui, muitas dessas ferramentas ficam para trás, não atendendo às demandas do MDS (Sistema de Dados Multicritério). A menos que haja um motivo convincente, recomendamos considerar alternativas às ferramentas corporativas legadas.

Machine Translated by Google

Moderno

É aí que entram em cena Fivetran, Stitch e Airbyte.
Projetadas em torno de "conectores", essas ferramentas podem interligar perfeitamente diversas fontes e destinos, impulsionadas por tecnologia de ponta e aproveitando o melhor do MDS.

Nativo

Nas duas primeiras soluções, partimos do pressuposto de que os dados devem ser movidos de uma fonte para outra — mas o quê?

E se você tivesse uma plataforma gerenciada que suportasse ingestão de dados de forma integrada? O Databricks, por exemplo, consegue ingerir dados nativamente de barramentos de mensagens e armazenamento em nuvem:

```
CRIAR TABELA DE STREAMING raw_data
AS selecionar *
FROM cloud_files("/raw_data", "json");

CRIAR TABELA DE STREAMING clean_data
COMO SELECT SUM(profit)...
A PARTIR de dados_brutos;
```

Embora não exista um tipo "certo" de solução declarativa, muitos se beneficiarão da redução de custos para construir e manter essas soluções, especialmente aquelas nativas do seu provedor/plataforma de nuvem atual.

Custo de construção/manutenção

As soluções declarativas são, em grande parte, automatizadas — é aqui que você obtém o melhor custo-benefício! Engenheiros dedicados cuidam do desenvolvimento e da manutenção dos conectores. Isso significa que você está delegando o trabalho pesado a especialistas. A maioria das soluções pagas inclui equipes de suporte ou gerentes de clientes dedicados, que oferecem insights e orientações personalizadas para suas necessidades. Esses especialistas provavelmente têm uma visão geral do cenário de dados e podem ajudá-lo a lidar com a ambiguidade em torno de problemas específicos de dados ou conectá-lo com outros profissionais da área.

Por exemplo, soluções como o Lakeflow Connect da Databricks oferecem uma combinação de conectores modernos que também são nativos da plataforma, permitindo que os usuários obtenham todos os benefícios de um conector fácil de configurar e totalmente gerenciado, sem precisar integrar uma solução adicional de outro fornecedor.

Machine Translated by Google

Extensibilidade:

A extensibilidade gira em torno da facilidade de expandir soluções existentes. Qual a probabilidade de um novo conector ser adicionado ao Airbyte ou ao Fivetran? Você consegue fazer isso sozinho, usando a mesma estrutura? Ou precisa esperar semanas, meses ou anos por uma equipe de produto? Usar o Stitch será suficiente ou você precisará de uma solução complementar? Lembre-se: gerenciar várias soluções pode aumentar os custos e complicar os fluxos de trabalho. Em soluções declarativas, a extensibilidade é fundamental. Ninguém quer adotar uma solução apenas para descobrir que ela atende a apenas 15% das suas necessidades.

Custo da mudança

O custo da mudança é onde as limitações das ferramentas declarativas se tornam evidentes. Frameworks proprietários e métodos específicos de CDC (Captura de Dados de Conteúdo) tornam a migração para outra ferramenta dispendiosa. Embora manter-se fiel a um fornecedor possa ser um compromisso necessário, é essencial levar isso em consideração ao avaliar soluções potenciais.

As abordagens **imperativas**

para ingestão de dados podem incluir taps Singer internos, funções lambda, modelos Apache Beam ou tarefas orquestradas por meio de sistemas como o Apache Airflow.

Normalmente, organizações maiores, com recursos substanciais, encontram maior valor na adoção de uma metodologia imperativa. Manter e dimensionar uma estrutura de ingestão personalizada e interna geralmente exige a expertise de vários engenheiros de dados ou até mesmo de um profissional dedicado.
equipe.

A maior vantagem das soluções imperativas é a sua extensibilidade.

Extensibilidade:

Por natureza, o imperativo é personalizado — isso significa que cada ponto de acesso e destino é adaptado às necessidades do negócio. Ao explorar opções de integração de dados, torna-se evidente que nenhuma ferramenta isolada atende a todos os critérios. Soluções padrão inevitavelmente envolvem concessões.

No entanto, com uma abordagem imperativa, existe a liberdade de projetá-la precisamente de acordo com as especificações desejadas. Infelizmente, sem uma grande organização dedicada à engenharia de dados, essa extensibilidade é incrivelmente cara para construir e manter.

Machine Translated by Google

Custo de construção/manutenção

Embora as soluções imperativas possam resolver problemas complexos e difíceis de ingestão de dados, elas exigem bastante trabalho de engenharia. Basta dar uma olhada no diagrama de relacionamento de entidades do Stripe para perceber que isso pode ser incrivelmente demorado. Além disso, a natureza evolutiva dos dados — como mudanças no esquema ou a descontinuação de uma API — pode amplificar a complexidade. Gerenciar uma única fonte de dados é uma coisa, mas e quando você precisa escalar para várias fontes?

Um sistema verdadeiramente resiliente e imperativo deve incorporar as melhores práticas de design de software, enfatizando modularidade, testabilidade e clareza. Negligenciar esses princípios pode comprometer os tempos de recuperação do sistema e dificultar a escalabilidade, afetando, em última instância, as operações comerciais. Portanto, sugerimos que apenas empresas com uma infraestrutura robusta de engenharia de dados considerem a adoção de um modelo totalmente imperativo.

Custo da mudança

A transição de uma solução imperativa para outra nem sempre é simples, dada a potencial incompatibilidade entre os formatos de diferentes fornecedores. No entanto, em uma perspectiva mais positiva, plataformas baseadas em frameworks comuns, como o Singer, podem apresentar maior compatibilidade, oferecendo potencialmente uma transição mais suave em comparação com ferramentas puramente declarativas, como o Fivetran ou o Airbyte.

Soluções híbridas:

Encontrar o equilíbrio certo na integração geralmente significa adotar uma abordagem híbrida. Isso pode envolver o uso de ferramentas como o Fivetran para a maioria das tarefas de integração, enquanto se criam soluções internas para fontes exclusivas, ou a opção por plataformas como Airbyte/Meltano e a criação de componentes personalizados para fontes de dados não suportadas.

Contribuir para projetos de código aberto também pode ser recompensador em um ambiente híbrido. Embora não sejam isentos de falhas, conectores híbridos, como os encontrados nos taps da Airbyte ou da Singer, se beneficiam do amplo suporte da comunidade. Notavelmente, as contribuições da Airbyte para o setor influenciaram positivamente a dinâmica do mercado, levando concorrentes como a Fivetran a introduzir planos gratuitos. Também incentivamos uma abordagem proativa na exploração de bibliotecas e ferramentas emergentes. Por exemplo, a dlt (data load tool) é uma biblioteca de código aberto que demonstra grande potencial.

Machine Translated by Google

Considere a integração de dados como a escolha de um automóvel. Nem toda tarefa exige a potência de um carro de Fórmula 1. Na maioria das vezes, o que se precisa é de um veículo confiável e versátil. No entanto, embora um Toyota atenda a 99% das necessidades, você não encontrará uma Sienna em uma corrida de F1. A estratégia ideal? Confiar no veículo do dia a dia, mas garantir acesso a ferramentas de alto desempenho quando necessário.

Machine Translated by Google

CAPÍTULO 2

Transformação de dados

Enquanto a ingestão de dados simplesmente transfere dados do ponto A para o ponto B, a transformação de dados transforma dados brutos em informações valiosas por meio de vários estágios do ciclo de vida dos dados. Este capítulo explora as diversas linguagens, plataformas e tecnologias disponíveis para profissionais de dados executarem transformações de dados.

Veremos como garantir que as transformações de dados sejam realizadas de forma eficiente e bem coordenada, preparando o terreno para discussões mais detalhadas sobre eficiência, escalabilidade e observabilidade mais adiante neste guia.

O que é transformação de dados?

A transformação de dados é a arte de manipular e aprimorar dados para melhor atender usuários e processos. A transformação envolve pegar alguns dados, seja em estado bruto ou quase original, e realizar uma ou mais operações para aproximá-los do uso pretendido. Em um pipeline ETL, a transformação ocorre em vários pontos.

Os dados podem ser transformados no momento da ingestão e novamente em diversos pontos subsequentes. O objetivo da transformação de dados é convertê-los em um ativo, utilizando análise e ciência para criar algo de valor para o negócio.

A transformação pode ser tão simples quanto remover registros indesejados, por exemplo, filtrando-os, ou tão complexa quanto reestruturar completamente os dados de origem. A transformação existe em um espectro; há um número quase infinito de maneiras de transformar dados — é isso que torna as coisas interessantes!

Machine Translated by Google

Da mesma forma, a transformação pode ser orquestrada em qualquer linguagem com qualquer tecnologia, sem as limitações de provedores de nuvem, servidores ou fontes. Isso pode se manifestar como trabalhos Spark coreografados, funções lambda, fluxos de trabalho SQL ou talvez algo diferente — seja em Python, JavaScript, Rust, Scala, R ou (pasmem!) uma planilha. Embora defendamos a escolha de linguagens familiares e populares, seria negligente da nossa parte não mencionar a onipresença da prática de "transformar" dados.

Neste capítulo, forneceremos uma visão geral do que é transformação e como transformamos dados na prática. Em seguida, abordaremos a construção de uma solução de transformação, começando por padrões comuns de transformação e atualização de dados e continuando com as melhores práticas e considerações para transformação de fluxos de dados. Nosso objetivo é fornecer uma série de considerações para a construção de um sistema de transformação eficiente e equipá-lo com as ferramentas para adaptar esse conhecimento aos seus problemas específicos.

Onde estamos agora?

No cenário em constante evolução da engenharia de dados, a história da transformação de dados é uma narrativa de crescimento, simplificação e adaptação. Nos primórdios da internet, gigantes como Google e Yahoo foram pioneiros. Eles inovaram com frameworks de big data como Hadoop e MapReduce. No entanto, essas tecnologias eram apenas um pouco menos complexas do que álgebra linear de nível de pós-graduação.

Com a maturidade do setor, as ferramentas foram simplificadas e democratizadas. O Spark surgiu, oferecendo um mecanismo distribuído simplificado com APIs em Python, SQL e Scala. Pouco tempo depois, o Databricks surgiu com a missão de simplificar a implementação do Spark. Simultaneamente, os data warehouses ganharam força. BigQuery, Redshift e Snowflake surgiram como tecnologias escaláveis, eventualmente separando o armazenamento do processamento. Presto, Trino e Athena entraram na disputa, oferecendo desempenho de consulta extremamente rápido e escalando o SQL para uma linguagem de big data.

Combinando o poder dos data warehouses, a flexibilidade do armazenamento em nuvem e a disponibilidade de ferramentas de transformação, os dados nunca estiveram tão acessíveis. Hoje, os lakehouses surgiram como uma opção adicional para gerenciar dados em escala de forma unificada e com boa relação custo-benefício.

Machine Translated by Google

Como transformamos dados?

Como já mencionamos, a transformação de dados é um tema inerentemente amplo. Vamos tentar dar estrutura e abrangência à transformação de dados, discutindo ambientes de transformação, linguagens, frameworks e melhores práticas.

Ambientes

O local onde transformamos os dados geralmente determina como os transformamos. Aqui estão os ambientes mais comuns para transformação de dados:

A transformação de

dados no data warehouse é realizada utilizando SQL. Os data warehouses modernos possuem funcionalidades interessantes que permitem a transformação de diversos tipos de dados — materialização, lógica incremental e particionamento, para citar alguns exemplos. Os data warehouses sem servidor podem ser dimensionados dinamicamente para lidar com cargas de trabalho intensas, tornando-os ideais para a transformação de dados em grandes conjuntos de dados estruturados.

Lagos de dados

Os data lakes se destacam no armazenamento econômico de grandes quantidades de dados, tornando-os uma área privilegiada para preparação de dados. Ao contrário dos data warehouses, os data lakes não possuem recursos computacionais próprios, portanto, as transformações precisam ser orquestradas e executadas usando recursos externos. serviços.

Data lakehouses

podem combinar aspectos de data lakes e data warehouses para fornecer uma solução com maior flexibilidade, potencialmente a um custo menor. Utilizando serviços como o Apache Spark, os profissionais podem transformar dados em PySpark ou Spark SQL, resolvendo o problema de computação em um data lake. O Databricks SQL é um data warehouse que funciona sobre a plataforma Databricks lakehouse, permitindo os benefícios de um data warehouse sem servidor.

A escolha entre data lakes, data warehouses e lakehouses depende das necessidades específicas de um projeto, da experiência da equipe e da estratégia de dados de longo prazo da organização, cada um atendendo a diferentes facetas da transformação de dados com seu conjunto distinto de ferramentas e desafios.

Machine Translated by Google

Aqui estão algumas perguntas a serem consideradas ao selecionar onde suas transformações ocorrerão:

- Como se comparam os custos de manutenção de um data lake, um data warehouse e um lakehouse, especialmente no que diz respeito à complexidade das transformações de dados?
- Alguma plataforma se adapta melhor à estrutura da sua equipe ou organização? Qual é a melhor opção para manutenção, desenvolvimento ou expansão, considerando seus fluxos de trabalho atuais e previstos? • Quais são as considerações para segurança de dados e conformidade?

Ao escolher uma plataforma de dados, leve em consideração seu orçamento, as habilidades e a experiência de sua equipe, bem como as necessidades atuais e futuras.
O equilíbrio entre práticas consagradas e tecnologia de ponta é crucial para o sucesso a longo prazo.

Preparação

de dados: Entre transformações, os dados são frequentemente preparados — gravados em um estado temporário em um local adequado, geralmente armazenamento em nuvem ou uma tabela intermediária. Os dados preparados podem ser mantidos temporariamente ou indefinidamente, mas desempenham um papel crucial na transformação.

Quer você opte por uma abordagem que combine lago e armazém ou por uma implementação puramente em casa à beira do lago, como você irá armazenar os dados? A arquitetura do medalhão (Figura 2-1) é uma abordagem que descreve três "camadas" distintas: Bronze para o material bruto, Prata para a transformação da luz e Ouro para o material "limpo".

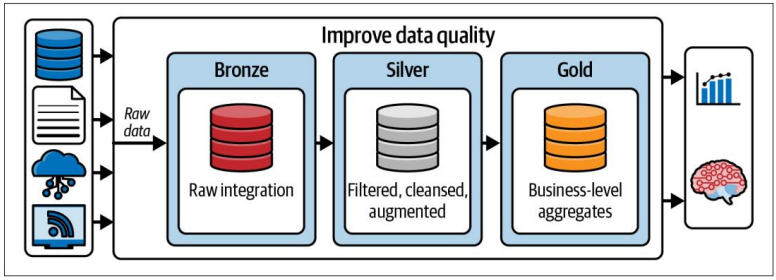


Figura 2-1. Uma arquitetura em medalhão promove três camadas distintas de preparação de dados.

Usando termos da arquitetura de medalhão, os dados em etapas geralmente seguem este padrão:

Machine Translated by Google

- Os dados em bronze são brutos e não filtrados, geralmente provenientes diretamente de fontes de integração. Por exemplo, suas tabelas brutas podem ser escritas diretamente a partir de uma API.
- Os dados "prata" são filtrados, limpos e ajustados. Você pode remover informações desnecessárias e irrelevantes, aplicar transformações para garantir a consistência e enriquecer os dados conforme necessário. • Os dados "ouro"

geralmente estão prontos para uso pelas partes interessadas e, às vezes, são agregados. Seus usuários (análise, engenharia) devem consultar as tabelas "ouro" na grande maioria das vezes, com necessidade ocasional de acessar as tabelas "prata". Geralmente, os ativos de produção (visualizações, relatórios) não devem acessar nada além das tabelas "ouro".

A jusante do seu lago, uma abordagem semelhante pode ser usada em um data warehouse, com tabelas de staging, curated e marts indicando os níveis de dados transformados. Esta é uma maneira consagrada de transformar dados e a abordagem que o dbt recomenda atualmente.

Assim, o padrão arquitetônico para a transformação em etapas consiste em dividir cada camada de armazenamento em "estágios" discretos de limpeza de dados. Integrado a essa abordagem está o aproveitamento da funcionalidade de armazenamento, como o Delta Lake. De qualquer forma, padrões como escrita-auditoria-publicação podem ser aplicados a qualquer estágio.

Linguagens.

É claro que a forma como transformamos dados é amplamente ditada pelas ferramentas à nossa disposição. Nossa principal escolha, a linguagem de programação, desempenha um papel fundamental nas bibliotecas, frameworks e métodos que utilizamos para chegar a uma solução. Aqui estão algumas das bibliotecas mais populares na área de dados:

Python

tem se mantido uma escolha firme no cenário em constante evolução da era digital, particularmente em ciência de dados e transformação. A Figura 2-2 mostra o domínio do Python na última década. No cerne das capacidades de ciência de dados do Python está a biblioteca Pandas, reforçada por uma gama de bibliotecas de entrada/saída para manipulação de dados. Historicamente, escalar o Python para grandes conjuntos de dados tem sido um desafio, muitas vezes exigindo o uso de bibliotecas como Dask e Ray. No entanto, avanços recentes marcaram uma evolução significativa no processamento de dados baseado em Python, que consideramos um renascimento na área.

Machine Translated by Google

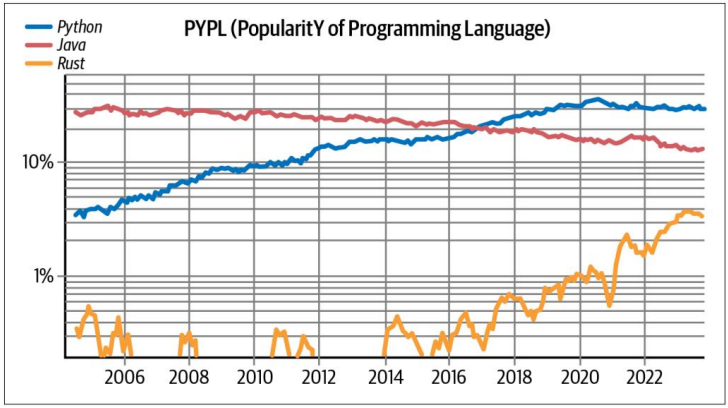


Figura 2-2. Embora Rust esteja em ascensão, Python continua a dominar o mercado (adaptado de PYPL); os valores do eixo y estão em escala logarítmica.

Ao transformar dados em Python, o primeiro passo é escolher uma biblioteca adequada, seguido pela seleção de um framework para orquestrar e executar a transformação. Recomendamos começar com o Pandas, conhecido por sua robustez. No entanto, bibliotecas emergentes como Polars e DuckDB também merecem ser consideradas. Este capítulo abordará diversas técnicas de transformação, e exploraremos a orquestração com mais detalhes no Capítulo 3.

SQL

ostenta o título de linguagem declarativa, um termo que agora aparece pela segunda vez (veja o Capítulo 1). Esse título implica que a linguagem se concentra mais em descrever os resultados desejados do que em detalhar o processo passo a passo para alcançá-los. Por exemplo, a instrução SELECT * FROM views.active_users não é código no sentido tradicional; é mais uma solicitação educada, dizendo: "Por favor, busque todos os usuários ativos".

Argumentamos que o SQL pode ser usado como uma linguagem declarativa ou imperativa, pois vimos algumas consultas bastante inovadoras (para o bem ou para o mal), mas o SQL ainda é inerentemente limitado pela falta de funcionalidades que linguagens como, digamos, Python, possuem.

Essa limitação inerente leva a maioria das transformações SQL a serem complementadas por uma solução de templates, facilitando a transformação SQL escalável. Linguagens como Jinja/Python (via dbt) e JavaScript (via Dataform) frequentemente complementam os fluxos de trabalho SQL.

Machine Translated by Google

A maioria das transformações SQL ocorre em um data warehouse ou utiliza o poder computacional do data warehouse para executar uma consulta.

Ferrugem

Rust é uma linguagem de transformação em ascensão, considerada por muitos como o futuro da engenharia de dados. Desenvolvida pela Mozilla, foi lançada em 2010. Sua principal vantagem reside na velocidade em comparação com Python e na tipagem forte, o que é benéfico para fluxos de trabalho em produção. No entanto, Rust é relativamente nova, e Python possui uma posição sólida devido ao amplo suporte da comunidade, ao rico ecossistema de bibliotecas e à ampla adoção em diversos setores.

Embora explorar Rust seja incentivado, para serviços de produção, é aconselhável optar por linguagens já consolidadas. Uma abordagem pragmática para Rust pode ser utilizar uma biblioteca Python escrita em Rust, como o Polars.

Estruturas

Os frameworks de transformação são mecanismos multilíngues para executar transformações de dados em máquinas ou clusters. Com a paralelização e a computação distribuída se tornando a norma, esses frameworks têm impulsionado a transformação em escala.

A vantagem de um framework é que existem múltiplas APIs para executar fluxos de trabalho, o que significa que as transformações podem ser manipuladas em diversas linguagens, como Python ou SQL. Abordaremos os dois mecanismos mais populares, Hadoop e Spark, embora você também possa considerar seu data warehouse como um mecanismo:

O Apache

Hadoop é um framework de código aberto que surgiu como uma solução inovadora para o gerenciamento de big data. Sua história, uso por grandes corporações e estado atual estão intimamente ligados à ascensão do Apache Spark e ao advento dos data warehouses sem servidor. O Hadoop ganhou força significativa em meados dos anos 2000, com gigantes da tecnologia como Yahoo, Facebook e, posteriormente, Google, adotando-o para gerenciar seus crescentes volumes de dados.

Embora o Hadoop tenha sido fundamental no processamento de dados em larga escala, ele apresentava limitações. Otimizado para processamento em lote, o MapReduce do Hadoop não era adequado para cargas de trabalho em tempo real ou iterativas.

Isso levou ao surgimento do Apache Spark no início da década de 2010, oferecendo

Machine Translated by Google

Processamento em memória, análises mais rápidas e suporte para diversas cargas de trabalho, incluindo processamento em lote, streaming e aprendizado de máquina.

Embora o Hadoop continue relevante, seu domínio diminuiu.
Muitas organizações têm migrado para arquiteturas de dados híbridas, construídas em torno de soluções baseadas em nuvem e data warehouses sem servidor.

O

Apache Spark é uma poderosa estrutura de processamento de dados de código aberto que revolucionou a análise de big data, oferecendo velocidade, versatilidade e integração com tecnologias essenciais. Sua história está intimamente ligada à necessidade de uma alternativa ao MapReduce do Hadoop, já que foi desenvolvido na Universidade da Califórnia, Berkeley, em resposta às limitações do MapReduce. A principal inovação do Spark são os conjuntos de dados distribuídos resilientes (RDDs), que permitem o processamento de dados em memória e cálculos mais rápidos.

O Apache Spark também possui uma forte ligação com a Databricks, empresa fundada por seus criadores originais, que fornece uma plataforma unificada de análise para engenheiros, cientistas e analistas de dados.
O Databricks simplifica a adoção do Spark e aprimora a colaboração, tornando-se um ator fundamental na análise de dados moderna.

Com APIs de alto nível em Java, Scala, Python e R, o Spark é acessível a praticamente qualquer profissional de dados, e a recente adição de Pipelines Declarativos do Spark ao framework expande ainda mais essa acessibilidade. Para começar a transformar dados no Spark, recomendamos considerar uma plataforma gerenciada, como o Databricks, pela sua baixa barreira de entrada e facilidade de uso.

Com a ascensão dos data

warehouses sem servidor, pode-se (com razão) questionar se os mecanismos de big data ainda são necessários. Os data warehouses sem servidor podem ser dimensionados verticalmente ou horizontalmente para atender à demanda prevista. Combinados com o uso apropriado de data lakes/lake houses e tabelas externas, eles podem ser uma maneira altamente eficiente de processar e transformar grandes quantidades de dados. Além disso, a simplicidade e a onipresença do SQL contrastam fortemente com a complexidade de outras tecnologias.

Machine Translated by Google

Em determinado momento, é provável que sua equipe precise usar o Spark (ou até mesmo o Hadoop), mas mecanismos de consulta como BigQuery, Databricks SQL e Redshift não devem ser descartados como soluções viáveis, especialmente nos estágios iniciais de uma equipe de dados. Com os recentes avanços na computação em memória (como MotherDuck e DuckDB), é provável que vejamos as capacidades de transformação dos data warehouses continuarem a se expandir. Fazendo jus à história, nunca duvide da durabilidade ou extensibilidade do banco de dados relacional e do bom e velho SQL.

Outras abordagens

Uma alternativa aos métodos tradicionais de transformação é o uso de

Plataformas de baixo código ou sem código. Devido às suas interfaces amigáveis, essas plataformas são particularmente adequadas para organizações com menos conhecimento técnico. Embora ofereçam acessibilidade, isso geralmente ocorre à custa de menor personalização, escalabilidade e controle, portanto, certamente não são para todos.

Depender exclusivamente de ferramentas de interface gráfica do usuário (GUI) pode dificultar a adaptação do software a necessidades específicas, levando a ineficiências ou lacunas de funcionalidade. Além disso, soluções baseadas em GUI podem ser menos transparentes e mais difíceis de controlar por versão, tornando desafiadora a colaboração com outros desenvolvedores.

Outros métodos de transformação mais a jusante — por exemplo, ferramentas de BI ou Planilhas Google — estão em grande parte fora do domínio da engenharia. Embora façam sentido para análise ou para certos stakeholders, recomendamos que a responsabilidade por essas soluções seja diretamente dos usuários, ou seja, dos analistas. Por quê? Essas soluções geralmente não têm capacidade de serem colocadas em produção — a falta de integração contínua/entrega contínua (CI/CD), controle de versão e transformações como código as tornam inadequadas para o desenvolvimento de software.

Construindo uma solução de transformação

Até agora, discutimos as peças do quebra-cabeça — linguagens, ambientes e estruturas para transformar dados — mas como juntamos essas peças para criar algo valioso?
A próxima seção abordará os padrões de transformação e as melhores práticas em dados estruturados utilizados em pipelines de dados modernos.

Machine Translated by Google

Padrões de transformação de dados

Nesta seção, apresentaremos um conjunto dos padrões de transformação de dados mais comuns, juntamente com exemplos de sua aplicação em um ambiente de produção. A transformação de dados é, em grande parte, um exercício de mapeamento de padrões — entender quais transformações existem e quando devem ser aplicadas é a maior parte do trabalho!

Enriquecimento

Isso implica aprimorar os dados existentes com fontes adicionais, por exemplo, acrescentando informações demográficas aos registros de clientes para fornecer um panorama mais completo. O enriquecimento pode envolver a junção de estruturas internas ou a obtenção de dados adicionais de fontes externas.

Exemplo: Sua tabela de pedidos brutos possui uma coluna de status que representa o status do pedido como um código interno. Este é um código não legível por humanos.

Código legível que torna a análise confusa. Você se junta a pedidos para enriquecer códigos de status: “1337” para “concluído”.

A junção

(ou união) envolve a combinação de dois ou mais conjuntos de dados com base em um campo comum, semelhante a uma operação JOIN em SQL. A junção é vital ao integrar dados de fontes distintas e desempenha um papel fundamental na arquitetura de dados. Quais tabelas de fatos e dimensões devem ser unidas para melhorar a análise? Quais devem permanecer separadas? Compreender os diferentes tipos de JOIN é essencial.

Exemplo: Ao conversar com sua equipe de análise, você percebe que os relatórios frequentemente analisam os dados de vendas pelo país de origem do pedido, mas esses dados não estão presentes na sua tabela de vendas limpa. Unir os dados de vendas aos dados do usuário permite que você inclua o país de origem, economizando tempo e evitando confusão para sua equipe de análise.

Filtragem:

Ao filtrar, você seleciona apenas os pontos de dados necessários para análise com base em determinados critérios. Isso pode reduzir o volume e melhorar a qualidade dos dados carregados no sistema de destino.

Machine Translated by Google

Exemplo: Um novo sistema não deve conter dados de anos anteriores a 2025, mas você encontra alguns registros incorretos. Você filtra os dados para eliminar os registros incorretos: SELECT * FROM purchasing WHERE date >= '2025-01-01'.

Estruturação:

Estruturar envolve traduzir dados para um formato ou estrutura necessária. Isso pode significar transformar documentos JSON em um formato tabular ou vice-versa.

Exemplo: Uma API de dados financeiros retorna dados JSON. Você descompacta e estrutura os dados e, em seguida, os adiciona a um armazenamento de dados Parquet no S3.

Conversão

A conversão consiste em alterar o tipo de dados de uma coluna ou campo específico, como converter uma string para o formato de data/hora ou um número inteiro para um número de ponto flutuante. Este é um dos tipos de transformação mais comuns, especialmente na conversão entre fontes de dados semiestruturadas e estruturadas.

Exemplo: Uma API upstream retorna um timestamp no formato de string “2011-01-01 00:00:00”, mas para facilitar a recuperação e visualização, você converte os timestamps para o formato correspondente em seu banco de dados. Uma conversão de data/hora mais avançada pode envolver timestamps Unix, que não podem ser simplesmente convertidos.

Agregação é o

processo de resumir e combinar dados, como calcular o total de vendas em um determinado período ou a média de um conjunto de valores. A agregação de dados é essencial para tirar conclusões a partir de grandes volumes de dados. Como engenheiros, muitas vezes tentamos antecipar a agregação e aproximá-la o máximo possível da análise.

Isso garante que os dados possam ser analisados no nível mais detalhado. O objetivo da agregação é nos permitir extrair insights de grandes volumes de dados e fundamentar decisões de negócios, criando valor a partir de nossos ativos de dados.

Exemplo: Você recebe dados em nível de milissegundos de dispositivos sensores de IoT implantados por sua equipe de serviços. Devido à quantidade incrivelmente grande de informações, você armazena um pequeno subconjunto desses dados em um determinado momento e os agrega em tabelas subsequentes, com base no segundo.

Machine Translated by Google

Anonimização é o

processo de mascaramento ou ofuscação de informações sensíveis em um conjunto de dados para proteger a privacidade. Como profissionais de dados, devemos fazer o possível para proteger a privacidade de nossos usuários, tanto por razões legais quanto éticas. A anonimização pode incluir a criptografia de e-mails ou outras formas de remoção de informações de identificação pessoal (PII) dos registros.

Exemplo: Os e-mails são criptografados e armazenados como um identificador único para impedir que informações confidenciais do cliente cheguem aos usuários de dados subsequentes. As informações são retidas em tabelas com acesso muito limitado.

A divisão,

que consiste em dividir uma única coluna de dados complexa em várias colunas, pode ser considerada uma forma simples de desnormalização.

Exemplo: Você deseja fornecer informações sobre os domínios de e-mail dos usuários, mantendo a privacidade dos dados. Você divide os e-mails em colunas de prefixo e domínio, descartando o prefixo para manter o anonimato dos dados, mas preservando o domínio.

Deduplicação: O

processo de deduplicação, parte da normalização de dados, consiste na remoção de registros redundantes para criar um conjunto de dados único.
A desduplicação pode ocorrer por meio de agregação, filtragem ou outros métodos.

Exemplo: Um fluxo de eventos ocasionalmente captura eventos duplicados (com identificadores únicos universais [UUIDs] idênticos) para uma única ocorrência. As partes interessadas concordam que o registro mais antigo deve ser mantido. Uma transformação é criada para descartar os UUIDs de eventos duplicados.

Padrões de atualização de dados.

Parte da transformação de dados envolve entender como atualizar os dados que já existem no sistema de destino. Aqui, apresentamos maneiras de atualizar dados para garantir processos de transformação sem problemas:

Sobrescrever

A forma mais simples de atualizar dados envolve a exclusão completa de uma fonte ou tabela existente e a sobrescrita com dados totalmente novos.

Machine Translated by Google

dados. Embora essa seja, sem dúvida, a forma mais simples de atualização de dados, ela pode rapidamente se tornar insustentável à medida que o volume de dados aumenta. Ainda assim, dada a sua simplicidade, pode ser um bom ponto de partida.

Inserir:

Em vez de sobrescrever dados, você pode optar por adicionar novos dados à sua fonte existente. Esse é o processo de inserção. A inserção de dados adiciona apenas novas linhas; nenhuma linha existente é alterada.

Um caso de uso ideal para inserção é aquele em que os novos dados são totalmente independentes dos dados antigos; por exemplo, uma tabela de transações ou dados do mercado de ações para um determinado dia. Nesses casos, simplesmente inserir novos registros incrementalmente é uma maneira rápida e fácil de manter as fontes de dados atualizadas.

Upsert,

abreviação de update and insert (atualização e inserção), é um padrão de atualização mais complexo, com aplicações para captura de dados de alteração, sessão e desduplicação, tornando-o particularmente útil, embora complexo.

Os dados alterados são identificados por um identificador único predefinido e, em seguida, os registros pertinentes são atualizados e/ou inseridos. Dada a complexidade inerente às transformações upsert manuais, plataformas como o Databricks possuem a funcionalidade MERGE , que simplifica drasticamente o processo.

Essa simplificação é apenas um dos muitos benefícios oferecidos pelo uso de uma estrutura de pipeline declarativa, que lida com muitas das complexidades, permitindo que você se concentre na lógica de negócios que está implementando.

O exemplo a seguir, da Databricks, demonstra o UPSERT de forma clara:

```
MERGE INTO people10m
USING people10mupdates ON
people10m.id = people10mupdates.id WHEN MATCHED
THEN UPDATE SET

    id = people10mupdates.id, firstName =
    people10mupdates.firstName, lastName =
    people10mupdates.lastName, birthDate =
    people10mupdates.birthDate, ssn = people10mupdates.ssn,
    salary = people10mupdates.salary
```

Machine Translated by Google

```
QUANDO NÃO HÁ CORRESPONDÊNCIA
  ENTÃO INSERIR
    ( id,
    nome,
    sobrenome,
    data de nascimento,
    número de
    segurança social, salário
  )
  VALORES
    ( pessoas10mupdates.id,
    pessoas10mupdates.firstName,
    pessoas10mupdates.lastName,
    pessoas10mupdates.birthDate,
    pessoas10mupdates.ssn,
    pessoas10mupdates.salary
  )
```

O conceito de exclusão de dados é frequentemente mal compreendido na engenharia de dados. Encontramos dois tipos principais de exclusão: "rígida" e "relativa". Uma exclusão relativa pode envolver marcar um registro como "excluído" (por exemplo, definindo o status como "excluído"), enquanto uma exclusão rígida significa remover completamente o registro do banco de dados.

No contexto de regulamentações de privacidade como o Regulamento Geral de Proteção de Dados (RGPD), existe uma tendência a favorecer a eliminação permanente dos dados. No entanto, a utilidade da exclusão lógica não deve ser ignorada.
Por exemplo, em uma empresa de software como serviço (SaaS) que gerencia ativos digitais, a exclusão lógica permite a criação de um registro histórico do status de um ativo. Isso pode ser crucial para manter a comunicação com os usuários, analisar encerramentos de contas e outros aspectos operacionais. Em contrapartida, a exclusão permanente elimina esses registros históricos, o que pode ser problemático em casos de recuperação de dados.

Melhores práticas

Embora a transformação possa parecer bastante diferente dependendo do sistema, alguns temas emergem como constantes. Considere estas boas práticas ao desenvolver sua solução de transformação, independentemente de onde ela esteja localizada:

Encenação

Já discutimos como você pode usar uma arquitetura de medalhão para armazenar dados em um data lake ou warehouse, mas vale a pena...

Machine Translated by Google

Reiterando: o armazenamento temporário deve ser considerado sempre que os dados forem transformados, para proteger contra a perda de dados (melhorar a capacidade de recuperação) e garantir um tempo de recuperação (TTR) baixo em caso de falha.

Idempotência é

um conceito fundamental que garante consistência e confiabilidade. Embora "idempotente" seja um termo importante, pode parecer um pouco complexo. No entanto, trata-se de um conceito simples: repetir uma ação várias vezes resulta em resultados consistentes. Nesse sentido, a idempotência é semelhante à "reprodutibilidade". Imagine que você execute um pipeline duas vezes sem novos dados de origem — os dados de saída serão os mesmos? A idempotência é crucial para lidar com falhas e garantir a consistência dos dados em sistemas distribuídos.

Normalização e desnormalização

Esses termos podem ser confusos para profissionais de dados iniciantes (eu mesmo fiquei confuso quando os ouvi pela primeira vez). A normalização envolve refinar os dados para o formato limpo e ordenado que esperamos. Isso pode se tornar complexo rapidamente, então vamos ser breves: dados normalizados são únicos e, às vezes, possuem uma chave primária.

A desnormalização é precisamente o oposto. Envolve a duplicação de registros e informações para tornar um sistema mais eficiente. Em grandes sistemas de dados, elementos redundantes podem ser utilizados para melhorar o desempenho de sistemas analíticos ou elementos subsequentes.¹

Incrementalidade:

No que diz respeito às atualizações de dados, a incrementalidade define se nosso pipeline é um simples INSERT OVERWRITE ou um UPSERT mais complexo. O volume, o poder computacional e o custo determinarão em grande parte se fluxos de trabalho não incrementais são possíveis. Existem muitos padrões predefinidos para a criação de fluxos de trabalho incrementais em ferramentas como dbt e Airflow. O Databricks também possui uma biblioteca robusta de recursos para pipelines incrementais.

1 Para uma análise mais aprofundada sobre normalização e desnormalização, consulte “Normalização de Dados Explicada: Como Normalizar Dados” e “Desnormalização de Dados: O Guia Completo”.

Machine Translated by Google

> Se você estiver carregando dados em um banco de dados, usar o comando MERGE em vez de INSERT pode fazer maravilhas. Se estiver carregando dados em um data lake, usar "processamento com estado" ou "marcadores", ou seja, manter o controle de onde uma transformação parou, pode ser muito valioso.

Transformação de dados em tempo real

Em nossa discussão sobre dados em tempo real no Capítulo 1, apresentamos as diferenças entre transformações em lote, microlote e fluxo contínuo. Aqui, observaremos simplesmente que as transformações em microlote podem ser transformações de fluxo contínuo eficazes quando a latência é suficientemente baixa.

A transformação de dados em fluxo contínuo refere-se ao processamento de dados em tempo real, à medida que são gerados ou recebidos. Transformações de fluxo contínuo verdadeiras (Beam, Flink, Kafka) podem introduzir um nível de complexidade para o qual a maioria não está preparada. A menos que seja absolutamente essencial, recomendamos uma abordagem de micro-lotes com algo como o Spark.

O Apache Spark, particularmente através de seus componentes PySpark e Spark SQL, se destaca nessa área ao oferecer transformações de micro-lotes/streaming. Essas transformações são mais simples de implementar em comparação com as transformações de evento único, que são mais complexas devido às restrições de processamento de dados em tempo real, como o uso de janelas.

O Spark Structured Streaming é uma aplicação de streaming popular entre os desenvolvedores, pois lida de forma eficiente com atualizações incrementais e contínuas. Os principais recursos incluem agregações de streaming, janelas de tempo de evento e junções de fluxo para lote, tudo facilitado pela API Dataset do Spark de maneira mais direta do que outros métodos.

Embora tecnicamente utilize um mecanismo de processamento em micro-lotes, o Spark Structured Streaming atinge latências tão baixas quanto 100 milissegundos, com tolerância a falhas de "exatamente uma vez", o que é suficientemente baixo para ser considerado "streaming". Além disso, o Processamento Contínuo, introduzido no Spark 2.3, pode reduzir as latências para até 1 milissegundo, oferecendo garantias de "pelo menos uma vez", aprimorando ainda mais sua capacidade de transformação de dados em fluxo contínuo.

O futuro da transformação de dados

A arquitetura de dados moderna está passando por um segundo renascimento, impulsionado por novas tecnologias que abordam as deficiências dos fluxos de trabalho de dados tradicionais. Simultaneamente, os avanços na IA estão remodelando não apenas a forma como trabalhamos, mas também os próprios sistemas que construímos.

Machine Translated by Google

Quase semanalmente, novas ferramentas e tecnologias surgem, redefinindo o significado de transformação de dados. Felizmente, os conceitos e padrões apresentados neste capítulo são atemporais — independentemente das ferramentas que você utiliza, é fundamental seguir estratégias consagradas para gerenciar dados e criar ativos limpos que gerem valor para sua organização.

O futuro da transformação de dados é simultaneamente semelhante e diferente da transformação atual — semelhante no sentido de que as estratégias, habilidades e comunicação necessárias para arquitetar um sistema eficiente não mudarão, e diferente no sentido de que as ferramentas serão aprimoradas e os processos automatizados... bem, pelo menos é o que esperamos.

Ferramentas e automações de última geração podem ser uma bênção e uma maldição, pois conferem um poder imenso aos profissionais. Mas, como diz o ditado, com grande poder vem grande responsabilidade.

É nossa função, como engenheiros, garantir que possamos entregar sistemas de transformação bem planejados e executados, com uma alta relação custo-benefício.

Machine Translated by Google

Machine Translated by Google

CAPÍTULO 3

Orquestração de dados

Embora já tenhamos discutido a ingestão (E, L) e a transformação (T), apenas arranhamos a superfície do ETL. Ao contrário da visão de pipelines de dados como uma série de etapas discretas, existem mecanismos abrangentes que operam em um meta-nível, apropriadamente denominados "correntes subterrâneas" por Matt Housley e Joe Reis em Fundamentos de Engenharia de Dados:

- Segurança
- Gestão de dados •

Operações de dados (DataOps)

- Arquitetura de dados
- Orquestração de dados
- Engenharia de software

Neste capítulo, exploraremos o gerenciamento de dependências e a orquestração de pipelines, abordando a história dos orquestradores, o que é importante para entender por que certos métodos de orquestração são populares hoje em dia. Apresentaremos um conjunto de opções para você orquestrar seus próprios fluxos de trabalho de dados e discutiremos alguns padrões de design comuns em orquestração.

Machine Translated by Google

Ao longo deste texto, discutiremos como um "orquestrador" historicamente se diferenciou de uma ferramenta de "transformação". Abordaremos os motivos pelos quais isso ocorreu e por que pode não ocorrer no futuro, embora ainda acreditemos que um orquestrador separado seja a abordagem preferencial.

O que é orquestração de dados?

Todo fluxo de trabalho, seja de dados ou não, requer etapas sequenciais: tentar usar uma prensa francesa sem aquecer a água só trará decepção, enquanto transformações de dados mal sequenciadas podem gerar uma tempestade muito mais amarga do que uma manhã sem cafeína (embora os problemas dos descafeinados não devam ser banalizados). Em dados, essas "etapas" são frequentemente chamadas de tarefas e "fluxos de trabalho", ou grafos acíclicos direcionados (DAGs), um termo que exploraremos em breve.

A orquestração é um processo de gerenciamento de dependências, facilitado pela automação. O orquestrador de dados gerencia o agendamento, o acionamento, o monitoramento e até mesmo a alocação de recursos. Os orquestradores são bem diferentes dos agendadores, que são baseados exclusivamente em tarefas cron. Os orquestradores, por outro lado, podem ser acionados por eventos, webhooks, agendamentos e até mesmo dependências dentro do fluxo de trabalho. A orquestração de dados fornece uma maneira estruturada, automatizada e eficiente de lidar com dados em larga escala provenientes de diversas fontes.

A orquestração visa, antes de tudo, garantir que os fluxos de trabalho produzam resultados precisos e oportunos. Um bom orquestrador também deve priorizar a eficiência, a escalabilidade e a velocidade, embora, como veremos em breve, as operações ocorram em grande parte fora do orquestrador.

Por que orquestrar?

A orquestração direciona os fluxos de trabalho para maior eficiência e funcionalidade, com um orquestrador servindo como a ferramenta que viabiliza esses fluxos de trabalho. Normalmente, os orquestradores acionam pipelines com base em um agendamento ou em um evento específico. Pipelines orientados a eventos são benéficos para lidar com dados imprevisíveis ou tarefas que consomem muitos recursos. Aqui está um resumo das vantagens de ter um orquestrador em seu conjunto de ferramentas de engenharia de dados:

Machine Translated by Google

Os orquestradores de

gerenciamento de fluxo de trabalho auxiliam na definição, agendamento e gerenciamento eficiente de fluxos de trabalho, garantindo que as tarefas sejam executadas na ordem correta, gerenciando as dependências.

Automação

Como engenheiros, devemos automatizar o máximo possível (ou viável). Os orquestradores podem ser usados para automatizar tarefas rotineiras, repetitivas e até mesmo complexas, economizando tempo e garantindo que as tarefas sejam executadas conforme o planejado, sem intervenção manual.

Tratamento de erros e recuperação

Os orquestradores geralmente vêm com mecanismos integrados de tratamento de erros e recuperação. Eles podem tentar novamente uma tarefa com falha, notificar a equipe ou acionar outras tarefas para corrigir o problema.

Monitoramento e alertas:

Monitorar fluxos de trabalho e receber alertas de falhas ou atrasos é crucial para manter pipelines de dados confiáveis. Os orquestradores fornecem essas funcionalidades.

Otimização de recursos Ao

gerenciar quando e onde as tarefas são executadas, os orquestradores ajudam a otimizar o uso de recursos, o que é crucial em ambientes com recursos limitados ou dispendiosos.

Observabilidade e depuração: os

orquestradores fornecem uma representação visual dos fluxos de trabalho, gerenciamento de logs e outras ferramentas de depuração, o que é essencial para solucionar problemas e otimizar fluxos de trabalho.

Conformidade e auditoria

Os orquestradores mantêm um registro de auditoria de todas as tarefas, o que é crucial para a conformidade com a governança de dados e outros requisitos regulamentares.

A utilização de um orquestrador é uma etapa estratégica para a construção de pipelines de dados robustos, eficientes e escaláveis, garantindo que os processos de engenharia de dados sejam bem coordenados, monitorados e gerenciados.

O DAG

O termo “grafo acíclico direcionado” é possivelmente o mais desnecessariamente complexo em engenharia de dados. Emprestado da teoria dos grafos (por algum motivo), ele é simplesmente usado para descrever uma “árvore” de execução de dados.

Machine Translated by Google

onde as “tarefas” são representadas como nós e as “dependências” como arestas (Figura 3-1). Essas árvores são:

Direcionado

A execução das tarefas segue uma direção específica de uma extremidade da árvore à outra, indicando a presença de dependências dentro do grafo.

Acíclico:

Não há ciclos em nossa árvore; ou seja, se executarmos a rota 1 ÿ 2 ÿ 5 na imagem da Figura 3-1, não executaremos 5 ÿ 1 no mesmo pipeline.

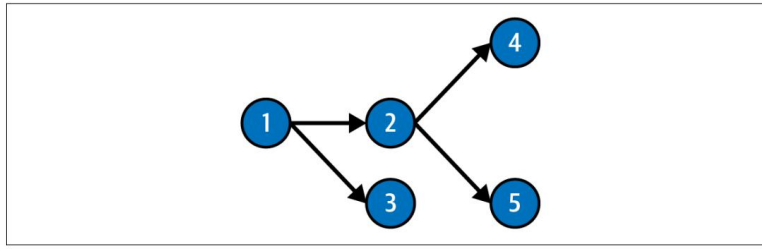


Figura 3-1. Um DAG muito simples

Os DAGs trazem ordem, controle e repetibilidade aos fluxos de trabalho de dados. Em contraste, minha vida é um grafo cíclico não direcionado — eventos caóticos parecem ocorrer repetidamente sem qualquer lógica ou razão. Portanto, sou grato, pelo menos, pelo padrão que os DAGs impõem à engenharia de dados.

O principal objetivo dos DAGs (Grafos Acíclicos Direcionados) é gerenciar dependências. Dependências podem se tornar muito complexas rapidamente. Antes que você perceba, estará trabalhando com algo que se parece com um diagrama de bifurcação, o que é apropriado, pois o caos provavelmente se instalará.

No domínio da engenharia de dados, os DAGs (Grafos Acíclicos Direcionados) são fundamentais para orquestrar e visualizar fluxos de trabalho. Eles servem como modelo para mapear uma sequência de tarefas, garantindo um fluxo de dados estruturado e previsível. Isso se torna indispensável no gerenciamento de fluxos de trabalho complexos, principalmente em equipes ou em grandes projetos.

Considere um pipeline de dados onde as tarefas são designadas para coleta, limpeza, transformação e, finalmente, carregamento de dados em um banco de dados. Um DAG (Grafo Acíclico Direcionado) serve como um roteiro claro, definindo a ordem dessas tarefas — garantindo, por exemplo, que os dados não sejam carregados prematuramente no banco de dados antes de passarem por limpeza e transformação.

Machine Translated by Google

Os DAGs foram popularizados por ferramentas de orquestração de fluxo de trabalho como o Apache Airflow, que facilita a criação, o agendamento e o monitoramento de pipelines de dados através da perspectiva dos DAGs.

Ferramentas de orquestração de dados

A trajetória das ferramentas de orquestração de dados reflete uma narrativa de notável evolução ao longo das últimas décadas. Inicialmente, os engenheiros de dados recorriam a soluções legadas ou improvisadas para orquestrar cargas de trabalho de dados — desde scripts Python personalizados até ferramentas de CI/CD adaptadas. No entanto, à medida que os cenários de dados se tornaram mais complexos, a demanda por ferramentas especializadas se intensificou, resultando na criação do Apache Airflow e do Luigi.

A trajetória da área está em constante evolução, impulsionada pela comunidade de código aberto e pelas demandas do Big Data. Embora o Apache Airflow tenha se consolidado como uma força dominante, existem diversas alternativas como Dagster, Prefect e Mage. Além disso, uma abordagem mais holística nos últimos anos oferece a orquestração como parte de uma solução unificada de ferramentas, como o Lakeflow da Databricks, que permite realizar ingestão, transformação e orquestração de forma integrada e imediata.
É importante escolher a ferramenta certa para cada tarefa, pois cada ferramenta tem seus pontos fortes e fracos. O futuro da orquestração de dados provavelmente será um ecossistema diversificado de ferramentas, cada uma com seu próprio nicho.

Como você provavelmente já deve ter

percebido, eu gosto de analogias. **Orquestração ,** literalmente, é o que o maestro de uma orquestra faz. Assim, os orquestradores de dados recebem um nome apropriado, mas as semelhanças vão além. Se os músicos da sinfonia são os membros da nossa pilha de dados, é o maestro quem garante que eles estejam tocando em harmonia e faz ajustes de andamento, fraseado ou repetições, transmitindo sua visão para a equipe.

Agora, o ponto importante aqui é que o maestro não está tocando um instrumento. Em algum momento, confundimos orquestração com plataforma de dados. Para ser justo, esse é um erro fácil de cometer. Por exemplo, o Airflow é bastante simples de instalar e aprender (e ainda mais fácil de implantar, graças à disponibilidade de opções gerenciadas) e, uma vez que tudo esteja configurado, você tem essa tela em branco perfeita para escrever… Python puro.

Machine Translated by Google

Em vez disso, usar orquestradores para o propósito para o qual foram criados, orquestrar, é a maneira mais eficaz de aproveitar sua vantagem competitiva. Mapeie suas dependências em serviços externos — Fivetran, dbt, Databricks e similares — e use seu orquestrador para acionar e monitorar esses serviços na ordem apropriada. Um maestro não consegue reger e tocar trombone ao mesmo tempo, então por que esperaríamos que nosso orquestrador o fizesse?

Um tema recorrente neste guia tem sido a necessidade de ferramentas híbridas que equilibrem frameworks declarativos e imperativos para proporcionar a flexibilidade do código com o rigor e a eficiência das melhores práticas de engenharia de software. Acreditamos que os orquestradores não são diferentes. Você pode construir um framework desse tipo por conta própria ou contratar um fornecedor/plataforma para fazer isso por você — isso depende muito da sua situação.

Características

Com isso em mente, podemos prosseguir para as considerações na seleção de um orquestrador. Aqui estão algumas características a serem levadas em conta:

Escalabilidade:

À medida que as necessidades de processamento de dados aumentam, os orquestradores ajudam a dimensionar os fluxos de trabalho para lidar com cargas crescentes. Discutiremos em breve os benefícios da orquestração em contêineres, mas considere se o seu orquestrador será capaz de escalar verticalmente (aumentando o número de tarefas paralelas) ou horizontalmente (aumentando o poder computacional de cada tarefa).

Como já mencionamos, o ideal é que o processo de orquestração dos seus dados seja separado do processo de transformação, portanto, a escalabilidade deve ser uma questão de lidar com lógica complexa e dependências, e não necessariamente com poder computacional. Pense em como seus fluxos de trabalho ficariam com 10 vezes mais DAGs, dependências e tarefas — seria gerenciável?

Reutilização de código e configuração

Ao contrário das áreas mais consolidadas do desenvolvimento de software, a engenharia de dados é notoriamente carente de boas práticas estabelecidas. Isso não quer dizer que elas não existam, apenas que não existirão se você não fizer um esforço genuíno para garantir sua implementação (escolhendo as ferramentas certas, assegurando a existência de frameworks, etc.).

Machine Translated by Google

Seu orquestrador deve promover a reutilização de código e configuração, simplificando o acesso a serviços semelhantes e a reutilização de tarefas comuns em diferentes pipelines. Para um bom exemplo de como isso é gerenciado no Airflow, consulte o SDK Astro. Qualquer ação repetida deve ser convertida em uma função ou classe.

Conexões

A orquestração, assim como a integração de dados, trata de conexões. O que isso significa? Se uma parte significativa dos seus fluxos de trabalho ocorre em uma plataforma específica, hospedar o orquestrador nessa mesma plataforma faz todo o sentido. Consequentemente, todos os principais provedores de nuvem oferecem uma versão hospedada do Airflow — Amazon (Managed Workflows for Apache Airflow), Google (Cloud Composer) e Azure (Azure Data Factory Managed Airflow).

Alternativas integradas à plataforma para o Airflow, como o Databricks Lakeflow Jobs, são indiscutivelmente mais adequadas para clientes com implementações do Databricks. Como os orquestradores servem para iniciar processos externos, garantir uma ampla biblioteca de conexões nativas, em vez de webhooks frágeis, proporcionará maior visibilidade das tarefas que você está orquestrando.

Suporte:

Orquestradores populares contam com forte suporte da comunidade e desenvolvimento contínuo, garantindo que a ferramenta permaneça atualizada com as tecnologias mais recentes e as melhores práticas. Ao investigar um orquestrador, como com qualquer ferramenta, pesquise com que frequência você pode esperar atualizações. Além disso, avalie a maturidade da ferramenta — sua equipe consegue lidar com a implantação de uma ferramenta em versão anterior à 1? Atualizações frequentes são viáveis ou uma solução mais estável é preferível?

Se você optar por uma solução proprietária ou paga, o suporte é igualmente importante. Sua equipe conta com engenheiros de soluções para auxiliar na implementação? Você pode contar com suporte para resolver problemas? Em alguns casos, essas equipes de suporte podem ser tão valiosas quanto consultores, dada a sua experiência com outras organizações que realizaram implementações semelhantes.

Machine Translated by Google

As áreas de observabilidade da pilha de dados moderna frequentemente se interligam. Embora os orquestradores não sejam necessariamente ferramentas de observabilidade, o seu deve fornecer informações sobre seus fluxos de transformação. Alguma tarefa falhou? Alguma foi repetida?

Pense no seu próprio processo. O que faz mais sentido para um sistema de alertas? Uma mensagem no Slack, um e-mail, uma carta manuscrita entregue por pombo-correio?[1] Certifique-se de que seu orquestrador seja compatível com o método que melhor se adapte à sua equipe e forneça a visibilidade necessária sobre o status dos seus fluxos de dados.

Opções do Orchestrator

Hoje, para implementar uma solução de orquestração, você pode:

Criar uma solução.

Não recomendamos esta opção pelo simples motivo de que a escala necessária para que ela faça sentido está no nível do Uber/Airbnb.

Adquira uma ferramenta pronta para uso. Essa pode ser uma opção valiosa para equipes que buscam abstrair o trabalho de DevOps e simplificar sua implantação.

Hospedar uma ferramenta de código aberto por conta própria: Se você tiver os recursos para hospedar por conta própria, essa opção pode ser uma ótima maneira de evitar a dependência de fornecedores. No entanto, é importante observar que a hospedagem própria traz consigo um conjunto totalmente diferente de desafios e problemas.

Utilize uma ferramenta incluída no seu provedor de nuvem ou plataforma de dados. Se você depende muito de uma plataforma específica, soluções como o Azure Data Factory ou o Databricks Lakeflow Jobs podem ser uma maneira simples e eficaz de orquestrar seus fluxos de trabalho.

Cada abordagem tem suas vantagens e desvantagens. Vamos discutir algumas opções, começando pela ferramenta que serve de base para diversas soluções nativas da nuvem (Amazon Managed Workflows, Google Cloud Composer, Azure Data Factory Managed Airflow) e que justifica sua própria categoria: Apache Airflow.

Nota : não recomendamos este método de alerta, a menos que você seja Johannes Kepler e O ano é 1612.

Machine Translated by Google

Desenvolvido pelo

Airbnb, o Apache Airflow se consolidou como a ferramenta de orquestração padrão, com uma comunidade ativa, ampla adoção e suporte de todos os principais provedores de nuvem. Ele continua sendo uma escolha popular graças à sua facilidade de adoção, implantação simples e onipresença no ambiente de dados.

É inegável que o Airflow não só foi fundamental no desenvolvimento da arquitetura de dados moderna, como também continua sendo uma ferramenta eficaz para orquestração. No entanto, observamos algumas falhas inerentes à ferramenta.

Concebido no início da década de 2010, o Airflow foi projetado para orquestrar, não para transformar ou ingerir. Apesar dos esforços louváveis da Astronomer, barreiras fundamentais nos fluxos de trabalho de transformação persistem no Airflow devido ao seu projeto original.

O Airflow é enganosamente fácil de configurar, mas muito mais difícil de implementar em escala e em produção. A falta de restrições permite erros simples, que muitas vezes vêm à tona sem a orientação de profissionais experientes.

Recomendamos fortemente que você explore as ferramentas mais recentes na área de orquestração e transformação para descobrir as opções disponíveis. Se o Airflow for sua única escolha, temos ótimas notícias: ele continua sendo uma excelente ferramenta de orquestração com amplo suporte da comunidade.

Outras ferramentas de

código aberto: Muitos orquestradores populares, incluindo o Airflow, são construídos em código aberto com a opção de serviço e suporte pagos e hospedados. Há diversas ferramentas mais recentes que estão inovando nesse segmento, com o Prefect e o Dagster liderando tanto em financiamento quanto em adoção. Várias ferramentas ainda mais novas, em versões anteriores à 1.0, têm ganhado destaque, como o Mage, o Keboola e o Kestra.

Optar por uma solução de código aberto pode ser atraente por diversos motivos, como o suporte da comunidade e a possibilidade de modificar diretamente o código-fonte. É claro que as ferramentas de código aberto dependem muito desse suporte para o seu desenvolvimento contínuo. Além disso, escolher um software em versão anterior à primeira provavelmente lhe proporcionará atualizações rápidas e novos recursos, mas com o risco de abandono do projeto e instabilidade.

Ao considerar uma solução, leve em conta seu histórico, suporte e estabilidade. Compare isso com os requisitos da sua empresa.

Machine Translated by Google

equipe, tanto agora quanto no futuro. Muitas ferramentas de código aberto oferecem a opção de hospedagem e suporte pagos. Se sua equipe estiver em condições de delegar o trabalho de DevOps de gerenciamento de uma instância hospedada, pode valer a pena pesquisar. Caso contrário, certifique-se de entender as opções de suporte (comunidade) das ferramentas escolhidas e a complexidade da implantação.

Ferramentas pagas e de código fechado

Assim como as opções gerenciadas de código aberto, as soluções pagas oferecem uma série de benefícios: suporte com acordos de nível de serviço (SLAs) predefinidos em relação aos tempos de resposta, possível consultoria/orientação de arquitetura e eliminação completa das dores de cabeça da implantação.

A principal desvantagem é a dependência do fornecedor em relação às ferramentas proprietárias.

Como a maioria das equipes de dados rapidamente se depara com dezenas ou até centenas de pipelines de dados, isso pode ser incrivelmente caro a longo prazo. Recomendamos avaliar minuciosamente qualquer solução proprietária paga quanto à reputação, funcionalidade e longevidade.

As plataformas de dados oferecem suas próprias soluções de orquestração, que são fortemente integradas a outros serviços da plataforma, como o Lakeflow Jobs. Esses orquestradores são ideais para equipes que utilizam exclusivamente uma plataforma específica e oferecem:

Melhoria na visibilidade:

Integradas diretamente à plataforma, essas soluções oferecem o máximo nível de visibilidade dos detalhes granulares das execuções do pipeline.

Compatibilidade máxima

Como já discutimos, o orquestrador existe para interagir com serviços externos. Uma opção integrada à plataforma é necessariamente compatível com outras ferramentas nessa plataforma.

Suporte ampliado:

Assim como outros serviços pagos, as ferramentas da plataforma oferecem suporte pago e engenheiros de soluções cuja função é ajudar sua equipe a ter sucesso. Esse serviço não deve ser ignorado e pode ser inestimável para uma nova equipe de dados que precisa de consultoria especializada em detalhes de implementação.

Machine Translated by Google

> Embora os orquestradores específicos de plataforma aumentem a dependência de fornecedores e tenham um custo elevado, eles podem reduzir drasticamente o trabalho necessário para manter e dimensionar sua solução. Esse valor não deve ser ignorado... e se você já utiliza uma plataforma como o Databricks, as soluções deles fazem bastante sentido.

Orquestrando SQL Até

agora, nos concentramos principalmente na orquestração de sistemas de dados inteiros, mas existe um subconjunto de transformação de dados que possui seu próprio conceito de orquestração: bancos de dados relacionais. Frequentemente, os dados são gravados em um sistema de origem e, em seguida, transformados sequencialmente com consultas e manipulações SQL complexas.

Antes da arquitetura de dados moderna, as equipes dependiam de infraestrutura personalizada para gerenciar essas transformações sozinhas. Hoje, usamos ferramentas como dbt, Lakeflow Declarative Pipelines, Dataform ou SQLMesh. Essas ferramentas são orquestradoras, pois avaliam dependências e condições, otimizando e executando comandos em um banco de dados para produzir o resultado desejado.

Essa sobreposição ganha importância no contexto da orquestração de dados.
Por exemplo, considere um DAG de fluxo de ar que:

1. Ingere dados em um estágio.
2. Transforma os dados de forma leve.
3. Aciona um projeto dbt.

O interessante é que o nº 3 é, na verdade, apenas um DAG — um que é executado em um data warehouse (Figura 3-2).

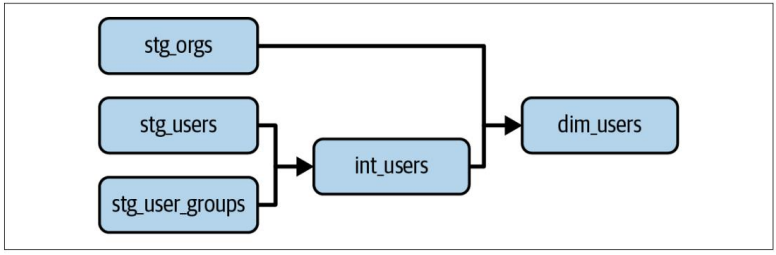


Figura 3-2. Um pipeline dbt — parece familiar, não é?

Machine Translated by Google

No entanto, essa estrutura indica uma possível limitação. Existe uma linhagem fora do orquestrador SQL e outra dentro dele, o que leva à necessidade de um mecanismo para observar os dados nessas camadas — da ingestão à transformação.

Este é um erro comum que muitas equipes de dados enfrentam, resultando frequentemente em uma desconexão entre as fontes e os dados limpos. Identificar erros nos dados subsequentes torna-se um desafio formidável: discernir se uma anomalia foi introduzida por um analista ou por um engenheiro de dados é como encontrar uma agulha em um palheiro.

O Lineage não só simplifica o processo de orquestração, como também promove uma visão mais abrangente da observabilidade dos dados, preenchendo a lacuna entre diferentes camadas de dados e garantindo um processo mais eficiente de detecção e resolução de erros. Esta é uma das razões para escolher um orquestrador de dados específico da plataforma: maior visibilidade, tanto entre quanto dentro dos fluxos de trabalho de dados.

Discutiremos a observabilidade mais detalhadamente no Capítulo 4, mas, por ora, basta mencionar que as estruturas de transformação baseadas em SQL são, em grande parte, orquestradoras, e encontrar uma orquestradora que ofereça observabilidade de ponta a ponta pode ser inestimável para fluxos de trabalho ETL.

Padrões de projeto e melhores práticas

Os padrões de projeto, quando empregados criteriosamente, podem aprimorar significativamente a eficiência, a confiabilidade e a facilidade de manutenção dos processos de orquestração de dados em um ambiente moderno de engenharia de dados.

Esses padrões existem fora das considerações mencionadas em “escolhendo um orquestrador”, já que a maioria pode ser realizada independentemente da sua escolha de orquestração. Deve-se notar, no entanto, que algumas soluções de orquestração tornam esses padrões significativamente mais fáceis — uma consideração importante na seleção:

Preenchimento de dados antigos: Se você estiver construindo um sistema de dados, provavelmente já existiam dados antes da sua implementação. Isso significa que você precisará preencher os dados antigos antes de começar a registrar novos dados. Muitos caem na armadilha do "pensamento pontual" e criam uma solução improvisada que não conseguem replicar.

Machine Translated by Google

Uma boa prática é que tudo o que for feito uma vez deve ser repetível.
Na orquestração, temos uma excelente oportunidade de incorporar lógica de preenchimento retroativo em nossos próprios pipelines — uma boa prática é perguntar: "Se esses dados desaparecessem amanhã, eu conseguiria recriá-los?"
Embora esse cenário exato possa não ocorrer, é provável que você precise adicionar uma coluna ou incluir algumas datas extras no futuro.

Em ferramentas como o Airflow, recomendamos a criação de DAGs que permitam definir uma data de início passada para acionar uma execução que recrie os dados históricos. Isso às vezes é mais fácil dizer do que fazer e exigirá pipelines idempotentes.

Idempotência.

Discutimos a idempotência no Capítulo 2, mas vale a pena reiterar: a idempotência, ou seja, garantir que a repetição de uma ação produza resultados consistentes, é essencial para uma engenharia de dados confiável.
Isso é especialmente importante ao executar um preenchimento retroativo — se você acidentalmente tiver datas sobrepostas, os dados serão duplicados? Podemos executar um pipeline para datas passadas, além das futuras?

Orquestração orientada a eventos

A orquestração orientada a eventos permite reagir a mudanças nos dados ou nos estados do sistema. O acionamento por eventos significa que seus dados estarão o mais atualizados possível ou ocorrerão no momento exato.
Por exemplo, se o seu data warehouse depender muito de uma tarefa de ingestão do Fivetran, você poderá acioná-la assim que essa tarefa for concluída.
Dessa forma, você pode evitar sintaxes cron complexas ou a possibilidade de uma tarefa de longa duração se sobrepor e causar problemas. O Lakeflow Jobs do Databricks, por exemplo, oferece acionamento com base na chegada de novos arquivos de dados ou em eventos de atualização de tabelas.

A orquestração orientada a eventos também pode ser usada para reduzir custos.

Um pipeline complexo com dados de origem pouco frequentes? Você pode criar um sistema que acione esse pipeline somente quando a fonte for atualizada.

Lógica condicional. O

interessante sobre dados e ETL, especificamente, é que as entradas estão sempre mudando. A questão não é "Será que vai dar problema?", mas sim "Quando vai dar problema e com que frequência?". Assim, um orquestrador deve ser capaz de lidar com lógica condicional — "se verdadeiro, então X; se falso, então Y" — para direcionar os fluxos de trabalho de dados.

Machine Translated by Google

A lógica condicional permite que sua equipe automatize ainda mais o processo ETL. Se uma exceção comum surgir em um pipeline, a adição de lógica condicional pode lidar com erros, falhas ou dados malformados, permitindo que sua equipe se livre do inferno do tratamento de exceções.

Concorrência.

Dada a existência deste livro, sabemos que ETL é difícil (ou pelo menos não muito fácil). É altamente provável que um ou mais dos seus pipelines sejam gargalos devido a uma grande quantidade de pequenas tarefas individuais. Por exemplo, você pode ter que ingerir milhares de pequenos arquivos CSV compactados. Percorrer cada um deles levaria horas, especialmente com o esquema de execução errado.

Em vez disso, podemos empregar concorrência, às vezes chamada de tarefas dinâmicas, para economizar tempo. Os orquestradores são obviamente capazes de executar pipelines concorrentes, mas a maioria pode distribuir as tarefas para serem executadas simultaneamente. A Figura 3-3 mostra um exemplo de um pipeline concorrente e dinâmico, executado no Apache Airflow.

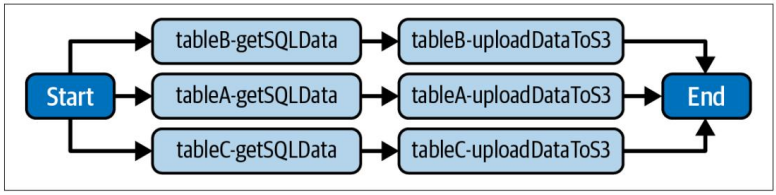


Figura 3-3. O mapeamento dinâmico de tarefas é uma forma de criar programaticamente tarefas executadas em paralelo no Apache Airflow.

Ciclos de feedback

rápidos. Os orquestradores são inerentemente complexos e frequentemente disparam ações difíceis de testar. Isso os torna notoriamente difíceis de desenvolver localmente. Já ouvimos a frase "a maneira mais fácil de parar uma instância local do Airflow é reiniciar minha máquina", que, embora seja um exagero, tem um fundo de verdade.

Recomendamos investir tempo e energia significativos na busca por uma ferramenta que permita falhar rapidamente e identificar erros com agilidade, alcançar paridade com uma instância implantada na nuvem e ter um ambiente local amigável para desenvolvedores e de rápida implementação.

Com a facilidade de implantação de infraestrutura (graças à conteinerização), também é possível implantar algo incrivelmente

Machine Translated by Google

Difícil de manter e expandir. Se você cair nessa armadilha, pode ser uma experiência muito, muito dolorosa.

Lógica de repetição e

contingência. Da mesma forma que precisamos estar preparados para o reabastecimento de tubulações, as repetições e falhas são inevitáveis. Em uma pilha de dados complexa, o bom gerenciamento de falhas garante a integridade dos dados e a confiabilidade do sistema, o que, por sua vez, facilita as operações de dados e reduz o tempo de inatividade.

Parte dos pipelines idempotentes consiste em lidar com falhas de forma a não omitir ou duplicar dados, mas sim criar um cenário para tentar novamente a operação ou ignorá-la e alertar as partes responsáveis. A maioria dos orquestradores permite definir diretamente um parâmetro de "tentativa" tanto no nível da tarefa quanto no nível do pipeline.

A lógica condicional pode ser combinada com a lógica de repetição/recurso alternativo para criar cenários que lidam com erros de forma adequada, reduzindo tanto o tempo gasto na triagem quanto os níveis de estresse em sua equipe de dados.

Execução parametrizada

Juntamente com a repetição e a lógica condicional, a execução parametrizada permite maior flexibilidade na sua orquestração. Adicionar parâmetros significa simplesmente permitir que o seu orquestrador aceite variáveis.

Essa pode ser uma maneira valiosa não apenas de lidar com diferentes casos, mas também de reutilizar pipelines para múltiplas finalidades. A execução parametrizada pode ser inestimável para facilitar o preenchimento de dados anteriores, por exemplo. Ao arquitetar um DAG (Grafo Acíclico Direcionado) que aceite uma data como parâmetro para buscar dados de uma API, por exemplo, você poderia criar uma estrutura simples para preencher um pipeline executando sequencialmente uma lista de datas a serem preenchidas.

Linhagem

refere-se ao caminho percorrido pelos dados ao longo de seu ciclo de vida. Dada a natureza visual do DAG, ele também é um ótimo lugar para entender as diferentes ações realizadas nos dados. Certifique-se de que sua solução de linhagem seja robusta, pois a linhagem é fundamental para depurar problemas e estender pipelines. Idealmente, busque uma linhagem em nível de coluna: uma visão em nível de coluna da jornada dos dados, da ingestão à análise.

Machine Translated by Google

A linhagem em nível de coluna está esclarecendo o caminho dos dados através dos pipelines de transformação, ampliando a rastreabilidade e a capacidade de depuração. Essa granularidade está prestes a se tornar um padrão da indústria em orquestração de SQL. Eu diria que a linhagem em nível de coluna deveria ser um requisito básico em qualquer nova implementação.
Um dos benefícios óbvios de uma solução de orquestração integrada à plataforma são os recursos avançados de linhagem.

Decomposição de pipelines

Uma boa estratégia para garantir pipelines organizados e legíveis é dividi-los em tarefas menores e mais gerenciáveis, que facilitem um melhor monitoramento, tratamento de erros e escalabilidade (Figura 3-4).

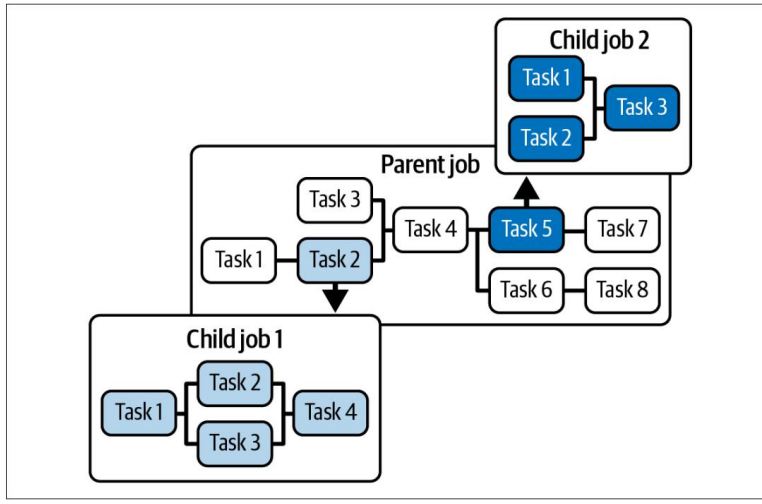


Figura 3-4. Um exemplo de decomposição de pipeline — dividindo pipelines em microsserviços e orquestrando-os com meta-DAGs.

Inspirando-nos na engenharia de software, considere os pipelines como microsserviços: uma função essencial dos microsserviços é sua autonomia e capacidade de operar independentemente uns dos outros. Busque construir DAGs (Grafos Acíclicos Direcionados) autônomos que possam ser executados em paralelo para mitigar dependências e falhas críticas. O design modular também facilitará a construção e a depuração de seus fluxos de trabalho.

Machine Translated by Google

O futuro da orquestração de dados

Você deve estar se perguntando por que este guia se concentra tanto na história e no futuro da arquitetura de dados moderna? Em um setor em rápida evolução, compreender as tendências é crucial — pode ser a diferença entre adotar uma ferramenta que se tornará obsoleta em poucos anos e uma que se consolidará no mercado. tração.

As ferramentas certas, avaliadas em termos de economia de tempo e recursos, podem impulsionar equipes de dados bem lideradas muito à frente de seus concorrentes. Embora tenhamos expressado reservas sobre a fusão de ferramentas de orquestração e transformação neste capítulo, a evolução das ferramentas de transformação, conforme discutido no Capítulo 2, pode mudar essa narrativa.

A infraestrutura conteinerizada, que facilita o escalonamento automático tanto vertical quanto horizontal, juntamente com uma nova geração de ferramentas que visam eliminar ferramentas complexas de big data (como MotherDuck, PyArrow e orquestradores modernos), pode reduzir a demanda por soluções de transformação robustas. Ferramentas de transformação híbridas como Prefect e Dagster merecem consideração séria.

A evolução na orquestração de SQL é um bom presságio para uma melhor observabilidade e monitoramento em nossos data warehouses (ou lake houses). Ferramentas emergentes como o SQLMesh estão prontas para desafiar players consolidados como o dbt, embora o dbt continue inovando e atualizando sua plataforma regularmente.

À medida que as plataformas amadurecem, soluções como o Lakeflow Jobs tornam-se cada vez mais atraentes do ponto de vista da facilidade de uso. O fato de essas soluções serem nativas de suas respectivas plataformas inspira confiança em sua funcionalidade e integração perfeita.

Manter-se atualizado em um ambiente que muda tão rapidamente é uma tarefa hercúlea, mas é por isso que tenho um emprego escrevendo guias como este. Independentemente disso, esses desenvolvimentos devem ser empolgantes, pois nos permitirão realizar nosso trabalho com mais eficiência e entregar o que todos buscamos: dados de qualidade de maneira oportuna e robusta.

Machine Translated by Google

Machine Translated by Google

CAPÍTULO 4

Problemas com o oleoduto e Solução de problemas

Como já mencionamos, o principal objetivo de qualquer processo ETL é gerar valor para o negócio. No entanto, esse objetivo é frequentemente prejudicado por sistemas frágeis, caracterizados por longos tempos de recuperação, o que leva à alocação excessiva de recursos para a resolução de problemas em vez de inovação e criação de valor.

Um pipeline ETL bem projetado, portanto, deve ser robusto e de fácil manutenção, incorporando características essenciais como alta qualidade de dados, tratamento eficiente de erros e identificação eficaz de problemas — todos componentes integrais da observabilidade do sistema. Independentemente da sua capacidade de engenharia, seus sistemas falharão — nem tudo está sob nosso controle!

Planejar para o fracasso significa projetar dutos que:

- São fáceis de manter e expandir — ou seja, permitem erros rápidos. triagem e desenvolvimento de novos recursos
- Fornecer maneiras automatizadas de lidar com erros em tempo real e recuperar-se de falhas.
- Incorporar uma estrutura de melhoria baseada na aprendizagem e na experiência.

Essa abordagem garante a facilidade de manutenção e se integra perfeitamente a outro aspecto crítico dos sistemas ETL: a escalabilidade.

Machine Translated by Google

À medida que nos aprofundamos no assunto, exploraremos como a manutenção de um sistema caminha lado a lado com sua escalabilidade, abordando desafios e estratégias para garantir que seu pipeline ETL seja resiliente e adaptável às necessidades de negócios em constante evolução.

Manutenibilidade

Em um capítulo sobre problemas e resolução de problemas, abordamos principalmente o tema da manutenibilidade — literalmente, a capacidade de manter as coisas que você já construiu.

Como engenheiros, às vezes buscamos soluções de forma exclusiva. Embora sejamos grandes defensores de produtos mínimos viáveis e do princípio 80/20, ao colaborar em sistemas para gerar valor para os negócios, a capacidade de manter e escalar (tema do Capítulo 5) esses sistemas é imprescindível. A incapacidade de manter um pipeline de vendas terá um custo direto e indireto para sua equipe. Embora o primeiro seja obviamente mais relevante, o segundo tem maior probabilidade de afundar seu navio de guerra.

Custo direto

> A maioria das pessoas está bem ciente do custo direto — se você quer algo caro, geralmente precisa desembolsar uma boa quantia e apresentar uma justificativa convincente. Quando as taxas de juros estavam próximas de zero e o capital de risco fluía como o Rio Grande, muitos ignoravam o custo de itens como computação em nuvem, armazenamento e DataOps. A eficiência começa com o design de padrões. Padrões ruins significam baixa eficiência em toda a organização.
>
> Um sistema ETL com baixa capacidade de manutenção acarretará um alto custo direto na forma de operações ineficientes e longos tempos de execução. Isso não apenas resultará em uma fatura alta do seu fornecedor, mas também em tarefas lentas e atrasos na entrega de dados pela sua equipe. A baixa capacidade de manutenção geralmente é resultado de uma infraestrutura fragmentada — um conjunto heterogêneo de ferramentas integradas é mais caro de manter. É por isso que muitas organizações optam por uma solução ETL unificada.

Custo indireto

> Os custos indiretos podem superar em muito os custos diretos. Criar sistemas que falham com frequência e exigem triagem constante pode ser desastroso para a alocação de recursos. O tempo e a energia da sua equipe são os recursos mais valiosos que você possui.

Machine Translated by Google

> Passar a maior parte do seu tempo de trabalho corrigindo DAGs, respondendo a alertas e apagando incêndios pode dar a impressão de que você está ocupado, mas geralmente se revela insustentável e improdutivo. As equipes vencedoras constroem sistemas eficientes que lhes permitem focar no desenvolvimento de funcionalidades e na democratização de dados.
> Sistemas ineficientes significam mais simulações de incêndio, mais noites sem dormir e menos horas dedicadas à construção de coisas que realmente importam.
>
> O principal argumento de venda do SaaS é que o benefício supera o custo, ou seja, o custo de desenvolvimento e manutenção de soluções internas. Se você faz parte de uma equipe com recursos ou experiência limitados, a solução mais fácil de manter pode ser uma comprada.
>
> É claro que essas interrupções não afetam apenas os engenheiros de dados — aqueles que trabalham em etapas posteriores da cadeia de suprimentos também são impactados. Isso significa uma perda de confiança por parte de
>
> Os custos indiretos de uma falha de dados, especialmente uma que exponha informações pessoais identificáveis, podem ser catastróficos para as partes interessadas, causar perda de receita de clientes e até mesmo prejudicar a reputação perante o público em geral.

Então, por que focar na manutenibilidade? Equipes vencedoras são construídas sobre sistemas manuteníveis.

Se o seu objetivo é impulsionar a sua organização com dados oportunos e eficientes, você precisará minimizar problemas e a necessidade de resolução de problemas, reduzindo os custos operacionais diretos e indiretos. No restante deste capítulo, discutiremos exatamente como você pode fazer isso por meio da observabilidade, da qualidade dos dados, do tratamento de erros e de fluxos de trabalho aprimorados.

Monitoramento e avaliação comparativa

Aqui, fazemos a distinção entre observabilidade e monitoramento/avaliação comparativa. A avaliação comparativa e o monitoramento de sistemas são essenciais — um subconjunto da observabilidade. Mas a observabilidade não se resume apenas à resolução de problemas e à manutenção. O monitoramento e a avaliação comparativa de nossos sistemas são necessários para minimizar problemas em dutos e agilizar os esforços de resolução de problemas.

Sistemas rigorosamente monitorados e com parâmetros de avaliação bem definidos são configurados de forma bastante eficaz como "observáveis", facilitando a melhoria da manutenção dos nossos sistemas de dados e reduzindo os custos associados a dados corrompidos. Sem monitoramento e alertas adequados, sua equipe pode nem perceber que algo deu errado! A última coisa que queremos é que as partes interessadas descubram um erro nos dados... ou pior, que o erro passe despercebido e resulte em decisões de negócios ruins.

Machine Translated by Google

Devemos monitorar os dados durante a ingestão, transformação e armazenamento, lidando com os erros à medida que surgem (de forma adequada, se possível) e alertando a equipe caso (e quando) algo dê errado. É aqui que as soluções unificadas de ETL se destacam, proporcionando uma visão integrada de todos os aspectos do seu pipeline.

A observabilidade não serve apenas para solucionar problemas. Ela também nos ajuda a escalar, como discutiremos no Capítulo 5.

Métricas

A seguir, apresentamos algumas medidas essenciais utilizadas para avaliar a confiabilidade e a utilidade dos dados dentro de uma organização. Essas métricas ajudam a garantir que os dados coletados e processados atendam a padrões específicos e cumpram seu propósito de forma eficaz:

Atualidade

refere-se à relevância e à atualidade dos dados em um sistema. É a medida de quão atualizados e recentes os dados estão em comparação com os eventos do mundo real que representam. Manter a atualidade dos dados é crucial para garantir que as análises, a tomada de decisões e outros processos orientados por dados sejam baseados em informações precisas e recentes. Os engenheiros de dados trabalham para projetar e implementar sistemas que minimizem a latência nas atualizações de dados, garantindo que as partes interessadas tenham acesso aos dados mais atuais para suas análises e operações.

As métricas comuns de atualização para um conjunto de dados incluem:

- O intervalo entre o registro de data e hora mais recente (no seu
- O atraso entre os dados de origem e o

seu conjunto de dados • A taxa de atualização, por exemplo,

por minuto, hora ou dia • Latência (o tempo total entre a

aquisição dos dados e a sua disponibilização)

Volume

refere-se à enorme quantidade de dados que precisa ser processada, armazenada e gerenciada dentro de um sistema. É um aspecto fundamental do tratamento de dados. Lidar com dados em grande escala apresenta desafios, como armazenamento eficiente, recuperação rápida e velocidade de processamento. Altos volumes de dados exigem infraestrutura e técnicas especializadas, como computação distribuída, processamento paralelo e compressão de dados.

Machine Translated by Google

As métricas de volume incluem:

- O tamanho de um data lake (gigabytes, terabytes, petabytes)
- O número de linhas em um banco de dados
- O volume de transações diárias em um sistema

Qualidade
: envolve garantir que os dados sejam precisos, consistentes e confiáveis ao longo de todo o seu ciclo de vida. A qualidade dos dados gira em torno de precisão, consistência, confiabilidade, pontualidade, integridade, segurança, documentação e monitoramento. Abordar esses aspectos garante dados de alta qualidade para a tomada de decisões e análises informadas.

Aqui estão alguns exemplos de métricas de qualidade de dados:

- Unicidade: existem linhas duplicadas no seu conjunto de dados? •
- Completude: quantos valores nulos existem? Eles são esperados? •
- Validade: os dados estão formatados de forma consistente? Eles existem dentro do intervalo correto, por exemplo, maiores que zero?

Métodos

Monitoramento implica a capacidade de visualizar tudo o que está acontecendo em sua infraestrutura e detectar erros em tempo hábil. Qualidade de dados significa implementar medidas rigorosas que melhorem a qualidade dos dados observados.

A seguir, apresentamos padrões e técnicas que você pode aplicar para melhorar diretamente a qualidade dos dados que está monitorando, com o objetivo de reduzir erros e tempo de inatividade:

Registro e monitoramento. O
: primeiro passo para depurar é verificar os registros. Mas para verificar os registros, é preciso que haja registros para verificar! Certifique-se de que seus sistemas estejam registrando dados detalhadamente. Para os sistemas que você desenvolve, torne o registro obrigatório. Defina e codifique as melhores práticas de registro, incluindo bibliotecas e o que deve ser registrado. Você não só precisa ser um excelente administrador de dados, como também um verdadeiro "lenhador de dados", um termo que consideramos muito mais lisonjeiro.

Machine Translated by Google

A

linhagem, em seu conceito simples — o caminho percorrido pelos dados ao longo de seu ciclo de vida — é uma das maneiras mais importantes de observar seus dados. Ter representações visuais e em código de seus pipelines e sistemas é fundamental para as operações diárias e economizará uma quantidade enorme de tempo na triagem e depuração de problemas.

Para que a linhagem seja útil, ela precisa ser completa e granular. Completa no sentido de que todos os sistemas sejam observados, incluindo a interconexão entre eles. Idealmente, sua linhagem estará no nível mais granular possível. Para dados tabulares, esse nível é o da coluna. Metadados em nível de coluna fornecem a visão mais granular possível. Isso aprimora a capacidade da equipe de identificar erros, simplifica fluxos de trabalho e melhora a produtividade... e a experiência de trabalhar com dados. Como mostrado na Figura 4-1, os exercícios de linhagem começam com a coluna.

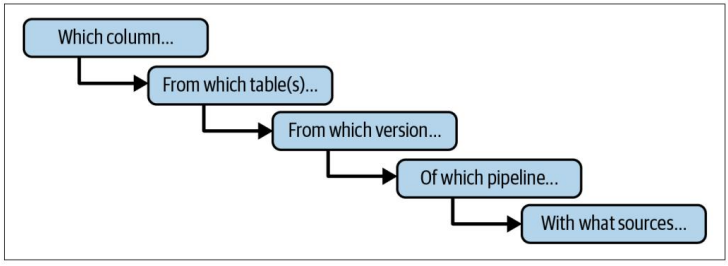


Figura 4-1. Uma avaliação dos princípios básicos da origem dos dados revela as complexidades da linhagem.

Um bom começo é implementar sistemas com soluções de linhagem autocontidas. Um sistema de linhagem mais completo pode monitorar todos os seus processos de dados. Plataformas gerenciadas são boas para isso, já que geralmente são autocontidas. Outras ferramentas, como Monte Carlo, podem fornecer insights sobre toda a pilha de tecnologia.

Detecção de anomalias.

O aspecto mais complexo dos dados é que não conseguimos ver tudo. A maioria dos nossos erros surge de stakeholders ou analistas que retornam resultados inesperados, o que nos leva à seguinte pergunta: "Que problemas estão à solta nos meus dados hoje?"

Machine Translated by Google

A detecção de anomalias é uma forma de saber que, dentro de um determinado limite, não existem dados anômalos. Os sistemas de detecção de anomalias funcionam por meio de previsões estatísticas básicas para analisar dados de séries temporais e retornar dados que se encontram fora de um determinado intervalo de confiança.

A detecção de anomalias pode ser uma ótima maneira de identificar erros que podem ter origem fora dos seus sistemas — por exemplo, uma equipe de processamento de pagamentos introduzindo um bug que subnotifica compras na Europa. Nenhum alarme será acionado, mas um sistema de detecção de anomalias direcionado para a tabela correta identificará o bug.

Diferenças

nos dados. A engenharia de dados contém uma dimensão adicional que a engenharia de software não possui: a qualidade dos dados. Alterar o código modifica os resultados de maneiras que podem ser difíceis de entender. Qual é o aspecto mais complexo dos dados?

As diferenças de dados são sistemas que relatam as alterações nos dados causadas por mudanças no código. Essas ferramentas informam sobre alterações em linhas e colunas, contagens de chaves primárias e outros efeitos específicos. Seu principal objetivo é garantir que os sistemas precisos permaneçam precisos.

Para soluções de comparação de dados, recomendamos ferramentas como o Datafold. Orquestradores SQL mais recentes, como o SQLMesh, também possuem funcionalidade de comparação de dados. A comparação de dados está intimamente ligada à CI/CD e é muito bem complementada por asserções e testes. Discutiremos tudo isso em breve.

Asserções

são restrições impostas às saídas de dados para validar os dados de origem. Diferentemente da detecção de anomalias, as asserções são muito mais simples. Por exemplo, você pode dizer: "Afirme que users.purchases contém apenas os preços da tabela plans.pricing." Se um valor que não existe em pricing aparecer em purchasings, você saberá que (a) um novo preço foi introduzido ou (b) há um erro em seu sistema.

Embora sejam manuais por natureza (as asserções exigem algum contexto de negócios e compreensão do que deveria ser), elas são uma das poucas maneiras de termos total confiança de que os dados não estão errados (pelo menos da maneira como os especificamos). Para soluções de asserção, confira bibliotecas como Great Expectations ou procure sistemas que tenham a capacidade integrada de definir asserções, como os recursos de expectativa do Lakeflow Declarative Pipelines.

Machine Translated by Google

Erros

Agora que discutimos a observação e a prevenção de erros, chegamos a uma verdade simples: independentemente da robustez da sua solução, ainda haverá erros! Portanto, o tratamento de erros torna-se extremamente importante, tanto para a sua sanidade mental quanto para a da sua equipe.

Além de lidar com erros, é importante recuperar-se de seus efeitos, sejam eles perda de dados ou tempo de inatividade. Uma parte importante da recuperação envolve retrospectivas, análises pós-incidente e reflexões sobre como evitar erros semelhantes no futuro.

Tratamento de erros

O tratamento de erros consiste em automatizar as respostas a erros ou condições de contorno para manter nossos sistemas de dados funcionando ou alertar nossa equipe de maneira oportuna e discreta. As abordagens a seguir detalham alguns métodos para processar erros de forma adequada e eficiente:

Lógica condicional:

Ao construir seus pipelines de dados, a lógica condicional pode ser útil para fontes não confiáveis ou inconsistentes. A capacidade de dizer "Se X, então A; caso contrário, se Y, então B" adiciona um componente poderoso às suas ferramentas de orquestração e transformação. Busque soluções que permitam o uso de lógica condicional.

Mecanismos de

repetição: Sistemas, mesmo os bem construídos, falham. Erros imprevistos podem causar timeouts pontuais de API ou outras anomalias. Por esse motivo, até mesmo um código que funcione bem pode apresentar erros. A lógica de repetição é importante em qualquer ferramenta de orquestração. Obviamente, para que isso funcione, configurações de repetição adequadas devem ser definidas. Certifique-se de incorporar uma estratégia de repetição que possa lidar com anomalias ou nuances nos dados, mas que não seja dispendiosa e não resulte em execuções infinitas.

Decomposição de pipelines.

Mencionamos anteriormente o conceito de modularidade no Capítulo 3, mas decompor pipelines em “microsserviços” é uma maneira eficaz de conter o impacto de erros. Considere a construção de DAGs e sistemas que exigem apenas as tabelas e conexões absolutamente necessárias.

Machine Translated by Google

Degradação controlada e isolamento de erros.

O isolamento de erros é viabilizado pela decomposição do pipeline — sempre que possível, os sistemas devem ser projetados para falhar de forma controlada. Por exemplo, você não gostaria que os dados de uso do seu produto estivessem a montante de um pipeline de relatórios financeiros, pois uma falha não relacionada poderia levar seu diretor financeiro a apresentar métricas atrasadas ao conselho, uma situação que terminaria bem para... ninguém.

A degradação elegante é a capacidade de manter funcionalidades limitadas mesmo quando uma parte do sistema falha. Ao isolar erros e decompor pipelines, estamos efetivamente possibilitando a degradação elegante. Você pode se deparar com um erro que apenas uma parte da empresa percebe, porque o restante dos seus sistemas funciona perfeitamente. Acredite, isso é muito melhor do que erros que todos notam.

O sistema

de alertas deve ser a última linha de defesa — receber alertas é necessariamente uma reação. Percebemos que algo ruim aconteceu e largamos tudo para corrigir o problema. Embora a resolução proativa de erros comuns seja a melhor opção, o inesperado acabará prevalecendo. Os alertas podem chegar por e-mail ou mensagem no Slack — preferimos o Slack, pois é altamente visível e os membros da equipe podem adicionar comentários com contexto útil.

Ao configurar alertas, esteja atento à fadiga de alertas. A fadiga de alertas refere-se a um número excessivo de notificações que diminui a importância de alertas futuros. Isolar erros e construir sistemas que se degradem de forma elegante, juntamente com notificações bem elaboradas, podem ser mecanismos poderosos para reduzir a fadiga de alarmes e criar uma boa experiência de desenvolvimento para sua equipe.

Recuperação :

Agora que temos uma visão clara dos nossos sistemas, estamos aplicando ativamente restrições de qualidade e temos métodos dedicados para lidar com erros. A peça final é construir sistemas para recuperação de desastres, que podem incluir perda de dados. A seguir, apresentamos alguns métodos e conceitos que ajudarão você a se recuperar após a inevitável falha:

Machine Translated by Google

Já mencionamos bastante o armazenamento temporário de dados (staging), mas um benefício adicional dos dados armazenados temporariamente é a recuperação de desastres. Com formatos baseados em Parquet, como o Delta Lake, e padrões como a arquitetura medallion, a possibilidade de restaurar dados (até certo ponto) permite viajar no tempo. Embora os dados armazenados temporariamente devam ser tratados como efêmeros, esse é um padrão importante para redundância.

Preenchimento de dados retroativos. É provável que você (a) queira dados de um período anterior ao início da execução do seu pipeline ou (b) tenha perdido dados que precisam ser preenchidos retroativamente. O preenchimento de dados retroativos é a prática de simular execuções históricas de um pipeline para criar um conjunto de dados completo. Por exemplo, o Airflow possui um comando de preenchimento retroativo que executa um pipeline para cada data entre duas datas.

Ao construir sistemas, leve em consideração a possibilidade de preenchimento de lacunas. Sistemas fáceis de preencher economizam bastante tempo quando algo falha. Busque ferramentas que suportem preenchimentos simples, pois a lógica de preenchimento pode se tornar muito complexa rapidamente.
Pipelines idempotentes também facilitarão sua vida. Verifique se o orquestrador de sua escolha oferece funcionalidade de preenchimento automático (backfill) pronta para uso.

Melhorando os fluxos de trabalho

Por mais que desejássemos que a melhoria dos processos estivesse totalmente sob nosso controle, não está. Nosso trabalho é inerentemente colaborativo — como verdadeiros "encanadores", fornecemos aos stakeholders os canais para dados internos e externos. Isso torna nosso trabalho inerentemente colaborativo.

Já mencionamos isso brevemente, mas a engenharia de dados é realmente uma questão de quando as coisas quebram, não de se vão quebrar. Mesmo nos melhores sistemas, surgem anomalias, erros acontecem e as coisas desmoronam. O Titanic é uma lição por um motivo, não é?

O fator decisivo passa a ser a sua capacidade de adaptação e resolução de problemas... e esse é o objetivo deste capítulo! Começar com sistemas que priorizam a resolução de problemas, a adaptabilidade e a recuperação é uma ótima maneira de reduzir dores de cabeça futuras. Nesta seção, apresentaremos algumas maneiras de você aprimorar continuamente seus processos.

Machine Translated by Google

Comece pelos relacionamentos.

Considere o seguinte exemplo: sua equipe de software altera, com certa regularidade, um esquema de produção sem aviso prévio. Isso não apenas interrompe seu processo diário de ETL com o conjunto de dados de produção, como também acarreta o risco de perda de dados, já que seu método de exportação não possui CDC (Credit Defaulting Data).

É compreensível que isso deixe sua equipe bastante frustrada. Além de perder tempo e recursos corrigindo esses problemas, eles parecem totalmente evitáveis. Antes de reagir de forma precipitada, pergunte-se: "A equipe de software está tentando reduzir a produtividade geral e gerar resultados piores?". Se a resposta for sim, recomendamos procurar um novo emprego, mas em 99,999% dos casos a resposta será "Não, claro que não".

Na verdade, se você se perguntar exatamente o que essa equipe está tentando fazer, provavelmente é a mesma coisa que você: fazer o melhor trabalho possível com os recursos disponíveis. Ótimo, agora você entende as motivações deles.

O próximo passo é entender os fluxos de trabalho deles e comunicar suas frustrações. A partir daí, você pode começar a criar um processo para melhorar a eficiência. Aqui estão algumas maneiras de garantir relacionamentos saudáveis por meio de uma abordagem estruturada e pragmática:

Acordos de Nível de Serviço (SLAs) são comuns no setor de provedores para garantir tempo de execução e disponibilidade, mas podem ser usados com a mesma eficácia dentro e entre equipes. Se você está enfrentando problemas com a qualidade dos dados, considere um SLA que defina formalmente aspectos como métricas de desempenho, responsabilidades, tempos de resposta e resolução e procedimentos de escalonamento, para que todos tenham uma compreensão clara do que é necessário para os dados recebidos.

Ao comunicar os requisitos de forma clara e atribuir responsabilidades, os SLAs podem ser uma forma surpreendentemente eficaz de melhorar a qualidade dos dados que estão fora do seu controle, simplesmente registrando alguns acordos por escrito.

Contratos de dados

Os contratos de dados ganharam força nos últimos anos como uma forma eficaz de governar os dados ingeridos de fontes externas.
Popularizados pelo dbt, os contratos são um tipo de asserção que verifica metadados (geralmente nomes e tipos de colunas) antes de executar parte ou a totalidade de um pipeline ETL.

Machine Translated by Google

Gostamos do termo "contrato" porque ele implica um acordo entre duas partes, como um SLA. Recomendamos primeiro definir um SLA, mesmo que seja um acordo informal, e depois implementar esse SLA na forma de contratos de dados em ativos externos. Se (e quando) um contrato retornar um erro, o SLA determinará exatamente de quem é a responsabilidade de resolver esse erro e com que rapidez a resolução deve ocorrer.

APIs

As APIs podem ser um método mais formal de fazer cumprir contratos e SLAs. Em certo sentido, o SQL em si é uma API, mas não há razão para que os dados internos não possam ser obtidos (ou fornecidos) por meio de uma API. As APIs são apenas um método de transmitir um conjunto esperado de dados, mas, quando implementadas corretamente, fornecem uma camada adicional de padronização e consistência à fonte. As APIs também oferecem controle de acesso mais granular, benefícios de escalabilidade e controle de versão, o que pode ser útil para conformidade.

Compaixão e empatia. Você pode

estar acostumado a ver esses termos em textos onde dbt aparece com inicial maiúscula e se refere a algo diferente de uma ferramenta de transformação, mas eles são tão importantes na engenharia quanto na psicologia. Compreender seus colegas de trabalho (e parceiros), suas motivações, dificuldades e fluxos de trabalho permitirá que você comunique suas preocupações de forma eficaz e explore seus incentivos.

Nesta era digital, é muito fácil adotar uma postura conflituosa ou presumir más intenções, especialmente a partir de reuniões por vídeo vagas e breves trechos de texto. Recomendamos que você se esforce para entender seus colegas de trabalho, seja por meio de conversas individuais, comunicações escritas mais longas ou reuniões presenciais, sempre que possível.

Alinhar incentivos

Para promover um progresso significativo, precisamos alinhar incentivos e resultados.1 Se a única missão da sua equipe for “construir muitas coisas”, pouco tempo será dedicado a tornar essas coisas resilientes e robustas.

1 Esta seção foi adaptada de um guia de Monte Carlo de Barr Moses, “12 Qualidade de Dados Métricas que REALMENTE importam”.

Machine Translated by Google

Definir indicadores-chave de desempenho (KPIs) em torno de métricas comuns de gerenciamento de incidentes pode ajudar a justificar o tempo e a energia necessários para realizar o trabalho corretamente:

Número de incidentes (N)

Contabilizar o número de incidentes ao longo de um determinado período de tempo fornecerá uma visão da frequência de dados incorretos, incompletos ou ausentes.

Tempo para detecção (TTD)

TTD descreve o tempo médio necessário para que um incidente seja detectado.

Tempo para resolução (TTR)

Essa métrica avalia a rapidez com que seus sistemas podem retomar as operações normais após uma interrupção. É uma medida direta da resiliência e da capacidade de recuperação do seu sistema.

Tempo de inatividade dos dados (N × [TTD + TTR])

Utilizando as três métricas acima, podemos chegar a um "tempo médio de inatividade de dados". Essa métrica resumida pode ajudá-lo a entender a gravidade das suas interrupções e a integridade dos seus sistemas.

Custo: A partir do tempo de inatividade, você pode calcular o custo dessas falhas.

O custo é altamente específico para sua equipe e organização. Recomendamos um cálculo de custos personalizado, levando em consideração as especificidades da sua implementação.

Melhorar os resultados. A

famosa citação de Thomas Edison, "Eu não fracassei, apenas descobri 10.000 maneiras que não funcionam", aplica-se perfeitamente ao processo de construção de pipelines de dados excepcionais e às estruturas que os suportam.
A jornada rumo à excelência nesta área é marcada por uma série de palpites fundamentados, experimentos e, crucialmente, pela capacidade de adaptação e correção de rumo diante dos desafios.

Se você acompanhou até aqui, sabe que ótimos sistemas são construídos sobre ótimas estruturas. Aqui estão algumas maneiras de iterar e aprimorar seus processos após falhas e ajustar seus fluxos de trabalho eficientes para torná-los excelentes:

Machine Translated by Google

Documentação

Se seus dados estiverem armazenados em um ambiente de teste e puderem ser restaurados posteriormente, a única coisa que resta é saber como corrigir o problema! Seja meticuloso e detalhista ao documentar os processos e o código do seu sistema. Embora possa parecer uma tarefa árdua, garantimos que será menos demorado (e estressante!) do que tentar reestruturar seu código durante uma falha.

autópsias

Em qualquer evento ou interrupção de grande porte, uma análise pós-evento pode ser valiosa para compreender a falha. Essa análise envolve refletir sobre o que deu errado e realizar uma investigação para entender o motivo.
As análises pós-evento, e a reflexão em geral, são excelentes maneiras de aprender, educar e crescer. Idealmente, suas análises pós-evento levarão a menos eventos que exijam recuperação.

Testes

unitários são o processo de validar pequenos trechos de código (os componentes de um sistema) para garantir que produzam os resultados esperados. Assim como qualquer outro sistema de engenharia, o código em um sistema de engenharia de dados deve ser testado unitariamente. Isso significa que qualquer código personalizado ou sistema sob medida que você criar deve ter testes para garantir que esteja produzindo os resultados desejados.

Esses testes são especificamente diferentes das asserções, já que os testes unitários verificam o código subjacente em vez dos dados de saída. Incorporar testes unitários ao seu código é uma excelente prática preventiva para minimizar erros futuros.

Embora muitas plataformas estejam adotando testes/asserções unitárias gradualmente, um número surpreendente de equipes de dados ainda opera sem eles. Recomendamos sua adoção em todos os sistemas de dados.

CI/CD

Integração contínua/implantação contínua (CI/CD) é um termo que se refere à forma como você integra e implanta seu código — ou seja, como as alterações (como solicitações de pull) são assimiladas, testadas e implementadas em sua base de código.

Machine Translated by Google

Na prática, CI/CD para engenharia de dados pode incluir várias coisas que já discutimos e algumas que ainda não mencionamos. Testes unitários, asserções, linting e comparações de dados garantirão uma base de código consistente, funcional e que permitirá a colaboração perfeita com outras pessoas para construir algo incrível (em escala).

Simplificação e abstração: Mais do que uma ferramenta, um padrão de projeto. Como engenheiro, você deve buscar simplificar e abstrair o máximo possível da lógica complexa do seu código. Isso não só facilita a colaboração, como também reduz a probabilidade de erros.

Ao escrever um código, pense: "Se eu não olhar para isso por seis meses, quão difícil será para mim entendê-lo?" Se algo quebrar em seis meses, você provavelmente estará analisando o problema sob algum tipo de pressão, então tenha isso em mente.

Sistemas de dados como código. As diferenças de dados abordam um conceito que ainda não discutimos: construir sistemas de dados como código. Ao versionar e codificar cada sistema, torna-se possível reverter alterações e retornar a estados anteriores. De certa forma, o uso de uma arquitetura de medalhão para criar ambientes de teste nos permite fazer algo semelhante a viajar no tempo.

Construir seus sistemas de dados com as melhores práticas de engenharia de software e implementar a lógica como código versionado melhorará drasticamente sua observabilidade, recuperação de desastres e colaboração. Desconfie de qualquer ferramenta que seja difícil ou impossível de versionar ou manipular por meio de código, mesmo que sejam apenas configurações JSON ou YAML.

Com as responsabilidades definidas, os incentivos alinhados e um conjunto completo de ferramentas de monitoramento e solução de problemas à sua disposição, você está pronto para começar a automatizar e otimizar seus fluxos de trabalho de dados. Reconhecendo que algumas tarefas não podem ser automatizadas e que sempre existirão casos extremos, a arte da engenharia de dados reside em equilibrar automação e praticidade.
Agora, com sistemas robustos e resilientes, estamos prontos para expandir.

Machine Translated by Google

Machine Translated by Google

CAPÍTULO 5

Eficiência e escalabilidade

Neste capítulo final, focamos nos aspectos cruciais da otimização e escalonamento dos pipelines de dados que desenvolvemos. Começamos definindo o que entendemos por “eficiência” e “escalabilidade” para delimitar nossa discussão.

Nossa jornada começa com a alocação de recursos, que depende de uma compreensão profunda do nosso ambiente operacional. Essa compreensão nos permite otimizar nossos processos com eficácia.

O capítulo culmina com uma discussão de duplo foco. Primeiro, exploramos o processo de colaboração, particularmente como escalar de forma eficaz em termos de tamanho da equipe e conjunto de habilidades. Segundo, aprofundamos a criação de uma experiência ideal para desenvolvedores, um fator chave na gestão eficiente de pipelines de dados.

Ao longo do capítulo, abordamos temas recorrentes, como considerações sobre ferramentas e plataformas, as vantagens e desvantagens de soluções gerenciadas versus soluções personalizadas e estratégias arquitetônicas para a criação de sistemas ETL superiores. Essas discussões visam fornecer uma visão abrangente da construção e manutenção de sistemas de dados eficientes e escaláveis.

Machine Translated by Google

Definição de Eficiência e Escalabilidade

Eficiência significa otimizar fluxos de trabalho para gerar valor comercial por meio de dados. Ela mede nossa capacidade de produzir resultados impactantes com os recursos disponíveis, abrangendo aspectos de código, serviços e trabalho em equipe. A medida definitiva de eficiência é o impacto produzido em relação aos recursos finitos utilizados.

Escalabilidade refere-se à capacidade de um sistema, rede ou processo de lidar com um volume crescente de trabalho, ou ao seu potencial de expansão para acomodar esse crescimento. No contexto da transformação de dados, trata-se da capacidade dos sistemas de processamento de dados de lidar com volumes crescentes de dados e tarefas de transformação mais complexas sem comprometer o desempenho ou a eficiência.

Imagine um sistema de transformação de dados que inicialmente processa dados de um pequeno site de comércio eletrônico. À medida que o negócio cresce, o mesmo acontece com seus dados — de milhares para milhões de clientes, transações e interações com produtos. Escalabilidade, neste contexto, significa que o sistema pode expandir suas capacidades de processamento de dados para gerenciar esse crescimento.

As equipes de dados frequentemente se deparam com o dilema entre preço e desempenho ao dimensionar sistemas de transformação de dados. Maior desempenho geralmente implica custos mais elevados — seja por meio de computação mais potente, soluções sofisticadas ou armazenamento em nuvem expandido.

Em dados, assim como na vida, o equilíbrio é fundamental. Embora seja mais fácil falar do que fazer, nosso objetivo é encontrar soluções que ofereçam o nível ideal de desempenho a um preço sustentável. A precificação baseada no uso, por exemplo, pode ser uma ótima maneira de começar a avaliar serviços, embora normalmente seja superada por assinaturas ou preços fixos em algum momento. As equipes de dados devem avaliar a relação custo-benefício das decisões de escalonamento, garantindo que estejam obtendo o desempenho ideal para seu investimento e não gastando demais com capacidade desnecessária.

ETL unificado

Outro fator a ser considerado, muitas vezes negligenciado, é a sobrecarga e a complexidade impostas pela necessidade de integrar múltiplas ferramentas. Uma integração mais complexa geralmente está associada a menor eficiência e dificuldade de escalabilidade. Imagine que seu pipeline ETL utilize uma ferramenta para ingerir dados e outra para transformá-los — se a integração entre as duas não for perfeita, você poderá encontrar gargalos ou falhas quando a ingestão escalar, mas a transformação ficar para trás. Isso é

Machine Translated by Google

> Onde ferramentas ETL unificadas são benéficas — ter tudo em um só lugar pode ajudar tanto na eficiência quanto na escalabilidade. Por exemplo, a Databricks lançou o Lakeflow, que inclui o Lakeflow Connect para ingestão, o Lakeflow Declarative Pipeline para transformação e o Lakeflow Jobs para orquestração — todos totalmente integrados e desenvolvidos para funcionar em conjunto de forma eficiente e em escala.

Como, então, podemos equilibrar eficiência, escalabilidade e custo-benefício?

Como discutimos no Capítulo 4, definir métricas claras em relação à qualidade, atualização e volume dos dados é crucial. Essas métricas podem justificar investimentos em otimização ou aumento de recursos computacionais. Por exemplo, se sua equipe precisa entregar dados a cada hora, mas o sistema atual é muito lento ou caro, essas métricas facilitam a argumentação em favor de otimizações ou ajustes orçamentários.

Métricas relacionadas à eficiência e escalabilidade também podem ser úteis. Aqui estão alguns exemplos de resultados-chave (KRs) relacionados à eficiência/escalabilidade:

- Aumentar a velocidade média de transferência em 25%
- Manter uma latência inferior a 200 ms em trabalhos de streaming
- Reduzir os custos de armazenamento de dados em 15%
- Aumentar a utilização de instâncias spot em 30% sem comprometer o desempenho

Os KRs podem ser obtidos definindo KPIs que se correlacionam com eles.
resultados:

Tempo de processamento
: O tempo necessário para processar um determinado volume de dados.

Capacidade de processamento
: A quantidade de dados processados em um determinado período de tempo.

Tempo de atividade do sistema
: A porcentagem de tempo em que um sistema está disponível e operacional.

Taxa de erro
: A frequência de erros em seus pipelines de dados

Alinhar os OKRs e KPIs da equipe com as expectativas das partes interessadas garante que os recursos sejam alocados adequadamente aos esforços de escalabilidade e otimização.

Machine Translated by Google

Compreender as ferramentas e os recursos à sua disposição, que abordaremos neste capítulo, juntamente com restrições como a atualização ou a qualidade dos dados, permite ajustar seus fluxos de trabalho para obter o melhor desempenho e custo.

Entenda seu ambiente

A escalabilidade na engenharia de dados está fundamentalmente enraizada na arquitetura. Estabelecer a estrutura e os fundamentos corretos é crucial para uma escalabilidade eficaz. Embora os avanços em dados tenham facilitado o início da construção, essa facilidade também traz o risco de criar sistemas que funcionam inicialmente, mas se tornam onerosos em termos de manutenção ou têm dificuldades para escalar.

Como diz o ditado, a retrospectiva é sempre perfeita, e cada implementação inevitavelmente levará a lições aprendidas e oportunidades de otimização. A chave para o sucesso na escalabilidade é uma compreensão profunda do seu ambiente, permitindo a construção de uma base robusta e duradoura para seus sistemas de dados.

Estruturas

A otimização de um sistema começa com uma compreensão completa do mesmo. Assim, antes de nos aprofundarmos na alocação de recursos, na melhoria da eficiência e na otimização dos fluxos de trabalho, precisamos entender a estrutura em que estamos operando.

Quanto melhor você compreender a mecânica de execução, mais eficazmente poderá otimizar suas tarefas. Se estiver usando o Pandas no Spark, por exemplo, dedique um tempo para estudar as melhores práticas; a documentação é sempre um bom ponto de partida. Se estiver executando SQL no BigQuery, entenda os planos de consulta e a ordem de execução. Mais uma vez, a documentação pode ser sua salvação.

Outros conceitos fundamentais de escalabilidade a serem considerados incluem: distinguir entre bancos de dados OLAP e OLTP, compreender os planos lógicos e físicos do Spark, diferenciar entre instâncias spot e sob demanda e entender como o sistema de computação distribuída escolhido — seja Databricks, Kubernetes ou até mesmo AWS Lambda — aloca recursos e escala.

Machine Translated by Google

Alocação de Recursos

Com uma base arquitetônica robusta, a escalabilidade dos seus sistemas de dados depende significativamente da alocação eficaz de recursos. Ter as ferramentas certas é apenas parte da equação; o que realmente importa é como você as utiliza. A chave é aproveitar os pontos fortes no contexto mais apropriado. Então, quais fatores devemos considerar na alocação de recursos? As próximas seções discutem alguns aspectos críticos a serem levados em conta.

Paralelização e concorrência:

Se você não estiver utilizando uma plataforma totalmente gerenciada e sem servidor para seus fluxos de trabalho de dados (o que é uma opção viável), geralmente terá certo controle sobre como seus pipelines de ETL são executados. No cenário atual, a computação distribuída, que aproveita o poder de clusters e nós, é predominantemente realizada em máquinas virtuais compartilhadas localizadas em data centers ao redor do mundo.

Sua configuração pode envolver um ambiente de execução auto-hospedado, como o Kubernetes, ou um ambiente compartilhado, como o Databricks. Cada opção apresenta seu próprio conjunto de considerações em termos de alocação de recursos, escalabilidade e manutenção.

Aglomerados

Ao gerenciar fluxos de trabalho ETL, uma decisão importante que você enfrentará envolve a configuração de clusters. Clusters são conjuntos de servidores que trabalham juntos como um sistema unificado, com cada servidor denominado "nó". Esses nós são responsáveis por executar e monitorar seus fluxos de trabalho.

Os clusters podem ser de uso geral ou específicos para determinadas tarefas; ou seja, podem ser usados para consultas ad hoc e análises gerais ou para tarefas específicas em seus fluxos de dados. O processo de configuração de clusters envolve decisões sobre o número e os tipos de nós. Essas escolhas afetarão fatores-chave como memória e poder de processamento, influenciando diretamente a capacidade computacional e os custos associados.

Uma compreensão detalhada da configuração de clusters permite a personalização dos seus fluxos de trabalho para diversas configurações, ou até mesmo a automação do escalonamento e da configuração. Isso possibilita alcançar um equilíbrio ideal entre custo-benefício e desempenho.

Machine Translated by Google

Nos parágrafos seguintes, exploraremos algumas técnicas eficazes para gerenciar a execução em cluster.

Instâncias spot versus instâncias de

cluster sob demanda: As instâncias spot oferecem uma solução com boa relação custo-benefício, geralmente com preços significativamente menores do que as instâncias sob demanda, tornando-as uma escolha prática para projetos com orçamento limitado. Elas são particularmente eficazes para tarefas que permitem flexibilidade e toleram interrupções, como processamento em lote ou análises de dados que não exigem resultados imediatos. No entanto, sua disponibilidade é variável, dependendo da demanda do mercado, e os provedores podem retirá-las com pouco aviso prévio, o que pode interromper tarefas em andamento.

Em contraste, as instâncias sob demanda garantem serviço contínuo, oferecendo confiabilidade e desempenho consistente. Elas são ideais para tarefas críticas para as operações e que exigem processamento em tempo real, onde a disponibilidade constante é crucial. No entanto, esse nível de confiabilidade e consistência tem um custo mais elevado em comparação com as instâncias spot.

A decisão de usar instâncias spot ou sob demanda deve ser baseada nas necessidades específicas de cada tarefa. Isso inclui considerar fatores como a importância da relação custo-benefício e a capacidade de lidar com possíveis interrupções com instâncias spot, em contraposição à necessidade de estabilidade e desempenho ininterrupto com instâncias sob demanda.

Pools

são utilizados em engenharia de dados para evitar o processo dispendioso de criação e destruição repetida de recursos. Especificamente, um pool se refere a um grupo de clusters que são mantidos ociosos e prontos para uso.

Optar por uma plataforma ou sistema que suporte o uso de pools pode aumentar significativamente a relação custo-benefício das suas soluções. Ao disponibilizar clusters prontamente, os pools podem reduzir o tempo necessário para iniciar clusters e escalá-los automaticamente.

Ao mesmo tempo, o uso de instâncias sob demanda para tarefas que exigem tempos de execução mais rápidos ou que possuem requisitos rigorosos pode aumentar a velocidade de desempenho. Essa abordagem permite um uso equilibrado dos recursos, otimizando tanto o custo quanto a eficiência em seus fluxos de trabalho de engenharia de dados.

Machine Translated by Google

Compartilhamento

de clusters: Clusters compartilhados oferecem uma abordagem econômica para a alocação de recursos em engenharia de dados. Ao permitir que vários usuários se conectem e executem suas cargas de trabalho simultaneamente no mesmo recurso computacional, os clusters compartilhados levam a economias de custos significativas. Eles simplificam o gerenciamento de clusters e facilitam a governança abrangente de dados, incluindo medidas precisas de controle de acesso. Apesar da implicação do termo "compartilhado", esses clusters fornecem um método seguro de redução de custos por meio da utilização eficiente de recursos.

Escalabilidade

automática Existem dois tipos de escalabilidade. A escalabilidade horizontal refere-se ao aumento do número de nós ou máquinas nos quais uma tarefa é executada; a escalabilidade vertical significa aumentar o tamanho ou a capacidade dos recursos existentes. Por exemplo, se um banco de dados estiver com pouco espaço de armazenamento, você pode escalá-lo verticalmente aumentando sua capacidade de armazenamento. Alternativamente, para melhorar o desempenho de uma tarefa grande e que exige muitos recursos, você pode optar pela escalabilidade horizontal adicionando mais máquinas. A Figura 5-1 fornece uma representação visual.

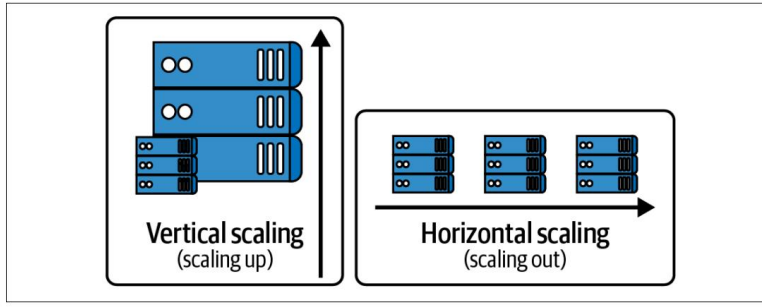


Figura 5-1. Escala vertical versus horizontal — ambas têm um momento e lugar apropriados.

O dimensionamento automático engloba as ferramentas e técnicas que ajustam automaticamente seus recursos para atender às flutuações da carga de trabalho. Um exemplo clássico são os sites de varejo dos EUA na Black Friday, quando há um pico significativo no tráfego da web. Manter recursos computacionais de alto nível durante todo o ano seria proibitivamente caro.

Em vez disso, o dimensionamento automático pode ser empregado para lidar de forma eficiente com esses picos de tráfego, mesmo quando são inesperados.

Atualmente, um número crescente de serviços em nuvem oferece recursos de escalonamento automático, que vão desde data warehouses até executores Spark.

Machine Translated by Google

O dimensionamento automático como um serviço gerenciado pode economizar tempo e recursos significativos, reduzindo a carga de tarefas de DevOps. No entanto, essa conveniência tem um custo, e o grau de utilização de serviços gerenciados de dimensionamento automático deve ser baseado nas necessidades específicas da sua equipe e no estágio de desenvolvimento em que se encontra.

Sem servidor

A computação sem servidor é um modelo em que os serviços de backend são fornecidos com base no uso real, em vez de uma alocação predeterminada de recursos. Embora os servidores ainda sejam parte integrante desse modelo, o aspecto distintivo é a estrutura de preços: as empresas que utilizam serviços de computação sem servidor são cobradas com base no seu uso, e não em uma largura de banda fixa ou em um número definido de servidores.

As soluções sem servidor, exemplificadas pelo BigQuery e pelos serviços da Databricks, como o Databricks SQL e o Lakeflow, oferecem a capacidade de escalonamento automático de acordo com as flutuações da carga de trabalho. Essa adaptabilidade reduz significativamente — ou, em alguns casos, elimina — a necessidade de ajustes meticulosos de recursos. Embora a computação sem servidor geralmente tenha uma estrutura de preços personalizada, a eliminação de certas manutenções a torna uma opção atraente para muitos.

Técnicas de Processamento de Dados

Independentemente de como seus recursos são alocados, você ainda precisará processar dados para obter a máxima eficiência. Aqui estão alguns conceitos a serem considerados ao construir seus pipelines, seja em um data warehouse ou em um data lake.

Processamento

incremental: Processar conjuntos de dados inteiros pode se tornar proibitivamente caro, especialmente à medida que o volume de dados aumenta. Embora truncar e recarregar seja uma abordagem direta, chega a um ponto em que se torna inviável ou excessivamente custoso em grande escala. O processamento incremental, que envolve adicionar apenas novos dados (INSERT) ou atualizar dados existentes enquanto insere novos (UPSERT), apresenta uma alternativa mais escalável.
A lógica de processamento incremental pode se tornar complexa rapidamente — recomendamos buscar ferramentas com padrões predefinidos para processamento incremental, como modelos incrementais do dbt, captura de dados de alteração ou Databricks MERGE.

Machine Translated by Google

Armazenamento de dados orientado a colunas

Para dados e análises, formatos e bancos de dados orientados a colunas oferecem melhorias substanciais de desempenho. Como a análise de dados frequentemente envolve a leitura por coluna, armazenar dados em formato colunar é inerentemente mais eficiente. Em contraste, bancos de dados OLTP tradicionais, como o PostgreSQL, são orientados a linhas, o que é adequado para cenários em que os dados são frequentemente gravados ou lidos uma linha por vez, como em produção. ambientes.

Atualmente, os armazenamentos orientados a colunas são os preferidos pelas equipes de dados e análises. Para obter o melhor desempenho analítico, considere utilizar formatos de dados como Parquet e Avro, e bancos de dados como BigQuery e DuckDB. Veja a Figura 5-2 para uma visualização dos dois conjuntos de dados. lojas.

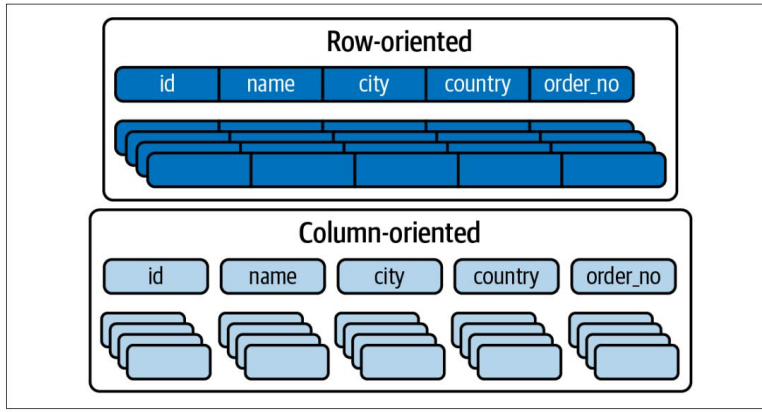


Figura 5-2. Bancos de dados orientados a linhas e colunas desempenham papéis fundamentais no desenvolvimento de dados e aplicativos, mas têm casos de uso muito diferentes.

Quando esses métodos de armazenamento colunar são combinados com particionamento (ou outra forma de segmentar dados), eles criam um padrão excepcionalmente poderoso e eficiente para o gerenciamento de dados. Bancos de dados como o Databricks SQL são construídos diretamente em formatos baseados em Parquet (Delta Lake), possibilitando otimizações de custo e melhorias de desempenho, como o agrupamento líquido.

Machine Translated by Google

Particionamento de

dados. O ato de "particionar" dados significa simplesmente dividi-los em partes. O particionamento é extremamente importante para o desempenho de leitura/gravação e não deve ser negligenciado. Usando formatos como Parquet e bibliotecas como PyArrow, o particionamento de dados pode ser simples.

A eficácia do particionamento torna-se particularmente evidente em conjuntos de dados colunares. Vejamos um cenário comum: um conjunto de dados contendo 100 dias de informações distribuídas em 10 colunas. Considere a seguinte consulta:

```
SELECT user_id FROM data WHERE date = '2024-01-01'
```

Em um conjunto de dados não particionado e baseado em linhas, essa consulta examinaria cada linha e coluna, resultando em tempos de execução lentos e custos computacionais aumentados.

No entanto, em um banco de dados orientado a colunas particionado por data, a consulta processa com eficiência apenas os dados necessários. Se assumirmos uma distribuição igualitária dos dados ao longo dos dias, acessar os dados de uma coluna para um dia específico em um banco de dados colunar particionado é mil vezes mais eficiente. Isso pode não fazer muita diferença em uma única consulta, mas ao acumular essa diferença em centenas de dashboards, consultas e pipelines, os benefícios podem ser percebidos rapidamente. A Figura 5-3 ilustra como o particionamento funciona para dividir conjuntos de dados e melhorar o desempenho.

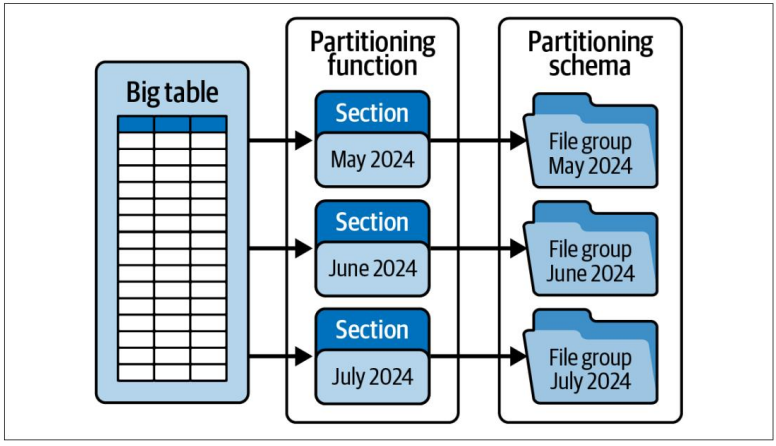


Figura 5-3. O particionamento de dados melhora tanto o desempenho das consultas quanto a eficiência do armazenamento.

Machine Translated by Google

Serviços gerenciados, que eliminam a necessidade de criar e manter partições, também merecem ser explorados. O particionamento pode ser um processo complexo que exige conhecimento do domínio e engenharia cuidadosa para ser criado e mantido. Sistemas que particionam, ordenam e otimizam conjuntos de dados automaticamente podem valer a pena.
custo.

Materialização

Materialização refere-se à criação de armazenamentos físicos de dados, ou seja, tabelas. Esse termo pode ser um pouco enganoso na era digital, já que "físico" é mais conceitual do que literal. Bancos de dados modernos geralmente suportam "visões", que são essencialmente representações virtuais de dados derivadas de consultas específicas. Por exemplo, você pode executar uma consulta como ` SELECT * FROM users WHERE status = 'active'` e definir isso como uma visão, `active_users`. Essa visão funciona como se fosse uma tabela, fornecendo uma maneira conveniente de acessar conjuntos de dados prontos para análise que precisam de transformação mínima dos dados de origem.

Uma “visão externa” refere-se a uma visão criada a partir de dados armazenados externamente — por exemplo, em um data lake. Uma tendência crescente envolve o armazenamento de dados semiestruturados em um data lake e a consulta direta desses dados como uma visão externa. Essa abordagem representa um “shift right” na engenharia de dados, onde a maior parte da transformação de dados é transferida para o data warehouse. Isso permite que analistas e outras partes interessadas interajam diretamente com dados semiestruturados, otimizando o processo.

Ao planejar transformações de dados, é crucial considerar cuidadosamente se deve-se usar tabelas, visualizações ou visualizações materializadas, pois cada uma tem suas próprias implicações e benefícios na arquitetura de sistemas de dados.

Eficiência do processo

É essencial ter um protótipo funcional ou uma solução de produção implementada. Uma vez que você tenha um sistema em funcionamento, a observabilidade torna-se fundamental. Isso significa ter a capacidade de monitorar e medir o desempenho da sua solução. Algumas perguntas a serem consideradas incluem: Quão bem a solução gerencia os recursos? Existem gargalos?
O que ocorre após a execução?

Machine Translated by Google

É crucial reavaliar suas suposições e decisões iniciais depois de obter informações valiosas observando seu sistema em funcionamento.
A engenharia, por natureza, envolve aprendizado contínuo e adaptação com base em novas percepções que não eram aparentes durante a fase de planejamento. A otimização é um processo constante, um ciclo contínuo de reavaliação de requisitos, busca por simplificação e exploração da automação.

Democratização de Dados (Engenharia de Dados) A

democratização de dados refere-se a tornar a tomada de decisões baseada em dados acessível a toda a organização, não apenas à equipe de dados. Comumente associada à análise de dados, geralmente envolve o fornecimento de ferramentas de análise de autoatendimento que permitem às partes interessadas explorar os dados de forma independente, sem depender de uma equipe de análise.

Para equipes de engenharia de dados, a democratização de dados envolve a criação de sistemas e estruturas que simplifiquem a criação de recursos e o desenvolvimento de tarefas ETL. Aqui está um exemplo:

> Um stakeholder precisa enriquecer dados com uma nova fonte de dados de API personalizada (ausente em um conector, como o Fivetran). A equipe de análise entra em contato solicitando um trabalho de ETL para extrair os dados da fonte, gravá-los em uma tabela de preparação e, em seguida, ingeri-los no seu data warehouse.

Muitos com quem trabalhei diriam: "Claro, podemos encaixar isso daqui a cinco sprints". Do ponto de vista do negócio, isso é uma situação em que todos perdem: aumenta o backlog de engenharia de dados e atrasa dados suplementares importantes. Cinco sprints e dois meses depois, a empresa implementa novos KPIs, a iniciativa é relegada a segundo plano e o pipeline fica sem uso. Quando estamos tão distantes do usuário (nossos stakeholders!), o "fim do dia" se transforma em "fim de dezembro" e as coisas se tornam muito menos eficientes.

Democratizar a engenharia de dados significa construir sistemas e plataformas de autosserviço. O objetivo desses sistemas é permitir que analistas ou outras partes interessadas realizem a maior parte do trabalho braçal, sem serem impedidos de se dedicarem ao trabalho que realmente precisam. A construção de um sistema segue este processo:

1. Identificar solicitações ou padrões comuns que os engenheiros de dados... frequentemente encontram
2. Avaliar a necessidade dessas solicitações. 3.

Simplificar o processo ao máximo.

Machine Translated by Google

4. Explorar ofertas de SaaS ou soluções personalizadas que possam ser usadas por usuários menos técnicos.

5. Simplificação e otimização contínuas 6. Colhendo os

benefícios

Esses sistemas ainda podem exigir revisões de código ou solicitações de pull, mas devem operar de forma amplamente autônoma. Isso pode envolver a criação de um repositório onde os analistas enviam SQL e YAML para gerar automaticamente um job do Airflow, ou o uso de soluções híbridas de GUI/código para o desenvolvimento interativo de pipelines.

Ferramentas que facilitam essa abordagem estão se tornando cada vez mais comuns (veja dbt Canvas e Lakeflow Designer). A capacidade das equipes de dados de implementar tais sistemas está se tornando um diferencial fundamental entre equipes que escalam com eficiência e aquelas atoladas em solicitações intermináveis — o diferencial entre uma equipe que escala e uma cujo quadro do JIRA permanece lotado até o ano seguinte.

A democratização de dados representa um aspecto singular da escalabilidade.
Isso alivia os gargalos nas equipes de dados, permitindo o autosserviço, seja por meio de plataformas de baixo código, sem código ou personalizadas internamente.
Sem recursos de autoatendimento, as equipes de dados correm o risco de ficarem perpetuamente sobrecarregadas por solicitações pontuais.

A eficiência e a escalabilidade **da**

experiência do desenvolvedor em sistemas ETL focam na otimização e no dimensionamento dos fluxos de trabalho de código, mas para as equipes de engenharia de dados por trás desses sistemas, trata-se de aprimorar a experiência do desenvolvedor (DevEx). Noda et al. definem as “dimensões principais” da DevEx como estado de fluxo, ciclos de feedback e carga cognitiva,[1] representadas na Figura 5-4.

1 Noda, A., et al. 2022. “DevEx: O que realmente impulsiona a produtividade? O desenvolvedor-abordagem centrada na medição e melhoria da produtividade.” Em Anais do Simpósio ACM SIGSOFT de 2022 sobre Fundamentos de Engenharia de Software (FSE) (pp. 32–47). ACM. https://dl.acm.org/doi/10.1145/3610285.

Machine Translated by Google

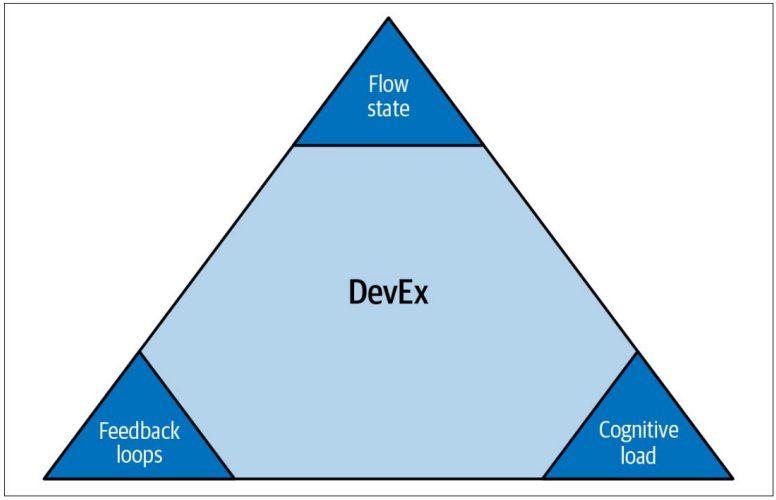


Figura 5-4. A experiência do desenvolvedor pode ser dividida em três áreas principais: estado de fluxo, ciclos de feedback e carga cognitiva.

Ciclos de feedback

podem ser definidos como a velocidade e a qualidade das respostas às ações. Considere o processo de desenvolvimento de um DAG (Grafo Acíclico Direcionado) no Airflow. Você consegue desenvolver localmente com facilidade? Ao fazer uma alteração, você consegue ver imediatamente como essa alteração afeta seu sistema? Seus dados? Acreditamos que os engenheiros de dados devem buscar uma experiência de desenvolvimento local semelhante. Engenheiros de front-end podem programar em uma janela e visualizar as alterações em tempo real em outra, implantando-as com alguns comandos simples. Como você pode construir seus sistemas para produzir uma experiência de desenvolvimento local similar?

Carga cognitiva é

a quantidade de processamento mental necessária para uma tarefa. Quão difícil é configurar seu ambiente de desenvolvimento local? Criar um novo modelo dbt? Compreender as tabelas, a infraestrutura e os metadados existentes em seu ambiente? Você costuma alternar entre sete ferramentas diferentes para realizar seu trabalho? Como engenheiros, devemos buscar simplificar e, assim, minimizar a carga cognitiva no desenvolvimento. Invista em sistemas que tornem a configuração e a construção o mais simples possível para que você possa se concentrar em trabalhos de grande impacto.

Machine Translated by Google

Estado de

fluxo. O fluxo é o estado mental de imersão, com sentimentos de foco, envolvimento e prazer. Todos já ouviram falar em estado de fluxo, e é o que todos buscamos. Como chegamos lá? Realizando um trabalho significativo e impactante, com ciclos de feedback curtos e baixa carga cognitiva. O primeiro componente é amplamente impulsionado pelo interesse, então não podemos ajudar nesse aspecto, mas ajustar seus sistemas para promover um desenvolvimento rápido em um ambiente de baixo estresse ajudará você a encontrar o fluxo.

Colaboração

Nossa última seção trata de colaboração. Grande surpresa: você conseguirá realizar muito mais com outras pessoas do que sozinho! No mundo da engenharia de dados, isso significa buscar soluções que sejam não apenas eficazes, mas também fáceis de entender, ampliar e utilizar por outras pessoas.

Infraestrutura como

código: Cada parte da sua infraestrutura deve existir como código versionado e testável. Isso permitirá que você aproveite tecnologias como Git e provedores como o GitHub para construir com o rigor de um engenheiro. Sim, você provavelmente se pegará vasculhando o Stack Overflow em busca daquele comando Git que você esqueceu, mas será muito melhor do que quebrar algo e ter que reconstruir tudo do zero.

Isso é especialmente importante em ferramentas de baixo ou nenhum código baseadas em interface gráfica.

Não as descarte completamente, mas certifique-se de que suas soluções tenham pelo menos um arquivo de configuração versionado e alguma forma de fazer backup/restaurar as configurações. A última coisa que você quer é uma ferramenta que exija 10 cliques para reconstruir um pipeline. Na primeira vez que você tiver uma falha, o pesadelo de reconstruir essas cem pipelines não será nada agradável.

Documentação e compartilhamento de

conhecimento. A documentação costuma ser vista como uma tarefa árdua, mas existe uma arte em comunicar e criar wikis eficazes. Tudo o que você fizer deve ser documentado e salvo para a posteridade (e para você mesmo). Deixe-nos poupar-lhe o trabalho; pode ser um pesadelo tentar lembrar-se daquela coisa que você construiu no ano passado ou tentar analisar o código de outra pessoa sem nenhum recurso. Documente o código que você escreve. Documente os dados. Garantimos que o esforço valerá a pena.

Machine Translated by Google

Conclusão

Eficiência e escalabilidade em engenharia de dados vão muito além da construção dos fluxos de trabalho mais rápidos e de melhor desempenho. Elas abrangem uma compreensão mais ampla das limitações de recursos, que todos inevitavelmente enfrentamos, bem como o papel vital da colaboração e a necessidade de fazer concessões.

Embora a expertise técnica em programação e implementação de algoritmos e estruturas de dados eficientes seja crucial, as habilidades interpessoais são igualmente importantes. Comunicação eficaz, documentação completa e gestão de equipe competente são componentes essenciais para o sucesso da engenharia de dados. Nosso objetivo com este capítulo, e com todo o guia, é fornecer a você um conjunto abrangente de habilidades que permita escalar seus pipelines de dados de forma eficaz e rápida — parafraseando um astronauta fictício igualmente ambicioso — “ao infinito e além”.

Machine Translated by Google

Conclusão

Obrigado por se envolver com nosso guia técnico, Compreendendo o ETL. Nesta jornada, exploramos as complexidades da ingestão de dados, aprofundamos as nuances da transformação de dados e desvendamos as complexidades dos fundamentos do ETL: orquestração, solução de problemas e escalabilidade. Ao longo do caminho, também lançamos luz sobre o cenário em constante evolução da engenharia de dados, com foco em metodologias consolidadas para a criação de conjuntos de dados limpos e confiáveis.

Embora tenhamos procurado abordar conceitos com relevância duradoura, é importante reconhecer que o campo da engenharia de dados provavelmente evoluirá significativamente nos próximos anos. No entanto, o processo fundamental de ETL — extrair dados de uma fonte, transformá-los e carregá-los em um destino — deverá permanecer como um pilar, independentemente dos avanços tecnológicos futuros ou das mudanças nos métodos de construção de sistemas de ETL.

Este guia forneceu uma visão geral fundamental da engenharia de dados, com o objetivo de ajudá-lo a identificar e preencher lacunas em seu conhecimento. O panorama dos dados pode ser vasto e complexo, mas compreender os componentes-chave e o contexto geral é crucial para navegar nele de forma eficaz.

Além dos aspectos técnicos, incorporamos temas como arquitetura de soluções e aprimoramento da experiência do desenvolvedor, ambos cruciais na engenharia de dados e em outras áreas. Enfatizar a importância de compreender seus colegas, usuários e os desafios que eles enfrentam não só o tornará um engenheiro mais competente, como também levará a soluções mais precisas e eficazes. Adotar uma filosofia de simplificação constante e tomada de decisões ponderada pode ser transformador em seu trabalho.

Machine Translated by Google

Por fim, encorajamos você a participar ativamente da comunidade de dados em geral. Seja contribuindo para projetos de código aberto, participando de conferências, escrevendo artigos ou compartilhando ideias nas redes sociais, seu envolvimento pode ser extremamente gratificante. Participar da comunidade não só promove o crescimento e o aprendizado pessoal, como também contribui para o avanço coletivo das práticas de engenharia de dados. Boa sorte — esperamos vê-lo(a) por aí!

Machine Translated by Google

Sobre o autor

Matt Palmer é engenheiro de experiência do desenvolvedor na Replit, com experiência em análise de produtos e engenharia de dados. Em seu tempo livre, gosta de escrever, fazer trilhas e musculação. Ele mora na região da Baía de São Francisco e passa a maior parte dos fins de semana explorando a Califórnia.